

Note to reader:

I submitted this internship progress report at the end of the first half of the internship(December 2023). I was told to submit this so that the supervising professor (who issued me my internship certificate, which I've also attached to my application) could decide whether I could do another 6 months at IIT Madras and build on the work I had done till then. I did receive an extension and continued the internship until May 15th, 2024. Unfortunately, I didn't keep track of my work and do not have another report like this, but I do have my codebase on my GitHub profile :

<https://github.com/kshitij-shekhar/image-to-image-translation-IITM-internship>

Hinweis für den Leser:

Ich habe diesen Praktikumsbericht am Ende der ersten Hälfte des Praktikums eingereicht (Dezember 2023). Mir wurde gesagt, ich solle ihn einreichen, damit der betreuende Professor (der mir meine Praktikumsbescheinigung ausgestellt hat, die ich auch meiner Bewerbung beigelegt habe) entscheiden könne, ob ich weitere 6 Monate am IIT Madras absolvieren und auf der bis dahin geleisteten Arbeit aufbauen könne. Ich erhielt eine Verlängerung und setzte das Praktikum bis zum 15. Mai 2024 fort. Leider habe ich nicht über meine Arbeit Buch geführt und habe keinen weiteren Bericht wie diesen, aber ich habe meine Codebasis auf meinem GitHub-Profil :

<https://github.com/kshitij-shekhar/image-to-image-translation-IITM-internship>

INTERNSHIP PROJECT - STAIN TRANSFER

KSHITIJ SHEKHAR
CCBR, IIT Madras
SGBC, IIT Madras
AHMEDABAD UNIVERSITY

INTRODUCTION

This project was based on the problem of stain transfer of brain imaging slices. When a particular part of the brain is sliced (20 microns in size), it's stained in order to enhance certain features of the slice and help neuroscience and histopathology researchers to study them in more detail.

The stain from a slice cannot be removed, this makes it harder to visualize that particular slice under different stains, and consequently, makes it harder to visualize the different features that each stain enhances. One cannot stain the same area of another brain with a different stain because every brain has nuanced structural differences.

This is the problem that the project I worked on seeks to solve. That is, to find a way to transfer a different stain onto an input stained brain slice in order to visualize the same slice under a different stain.

EXPERIMENTS TRIED - UNSUCCESSFUL TO SUCCESSFUL IN ORDER

1. Vanilla CycleGAN with patched(256x256) BCI dataset (Breast Cancer) images

I cropped the 1024x1024 BCI dataset images which consisted of HE and IHC images of 4 different types (they are divided into different types based on the intensity of the staining), with each type containing 4 subtypes which are divided based on the approximate region of the breast tissue that they stain. Unfortunately, they aren't representative, with each subtype having a different number of images across the 4 types (type_0 had 7-8 images of each subtype, while type_1, type_2 and type_3 had anywhere from 30-60 images of each subtype). This realization would dawn on me late into the internship, which helped in making a better model with a more representative dataset.

The architecture and flow of information in this model is as follows:

- There are two Generators : G_A and G_B. G_A takes images from domain A (say a IHC stained image) and outputs an image that will eventually look like it's from domain B. Vice versa happens to G_B. There are two Discriminators : D_A and D_B. These are simple binary classifiers, D_B takes as input, real images from domain B and generated images from G_A. For the real images, it tries to predict a high probability of being real (a number close to 1), while for the output of G_A it tries to predict a low probability of being real (a number close to 0). D_A does the same for outputs from G_B.

We trained this [model](#) on each type separately, with satisfactory results. The model had two ResNet Generators with 9 residual blocks and two 70x70 PatchGAN discriminators. We used Instance Normalization, as is recommended for image to image translation tasks, we also did not use dropout as a regularization technique([the original cycleGAN paper](#) does not use dropout). It was trained for 60 epochs with a learning rate of 0.0002 which would decay to 0 after 30 epochs.

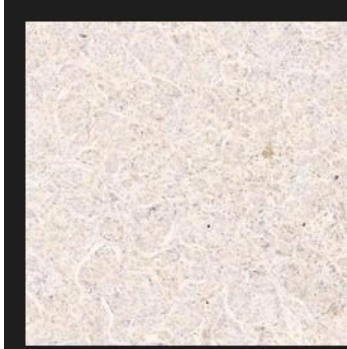
The losses that were built in to the model were Cycle Consistency loss(which helps ensure that the translated image can be fed through the same generator to output a reconstructed image that is as close as possible to the input image), the adversarial loss(standard in all GAN based architectures, is a minimax game between the Generator and Discriminator, where the Generator tries to generate an image that the Discriminator will have a hard time classifying as real or fake, while the Discriminator seeks to get better at distinguishing between a generated image and a real image), and an identity loss(which seeks to preserve the overall structure of an input image. If for example an image from domain A is passed to a Generator that transfers domain B to domain A, the identity loss makes sure that the image undergoes minimal transformations).

Here are the images of domain A were of IHC stains while domain B was HE stains. These are the intermediate images that were output during training after the latest epoch

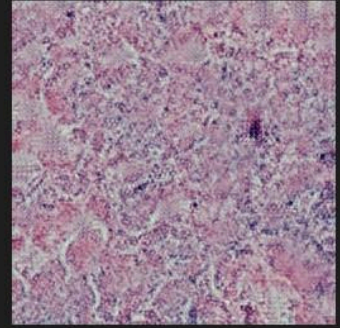


Source : IHC image(type_0)

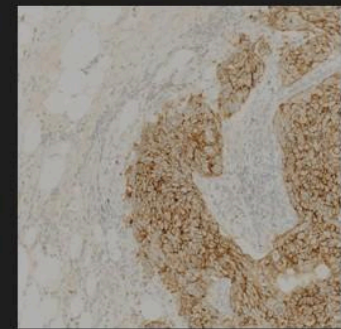
Output from the model



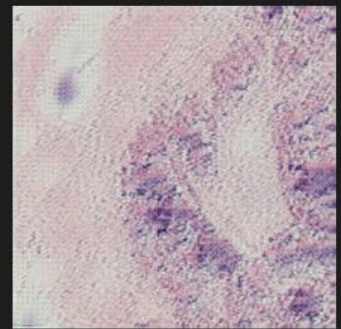
Source : IHC image(type_1)



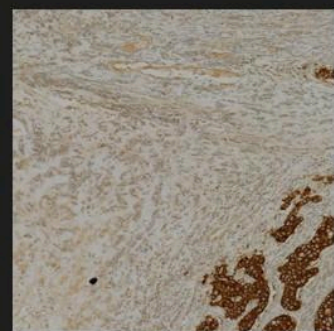
Output from the model



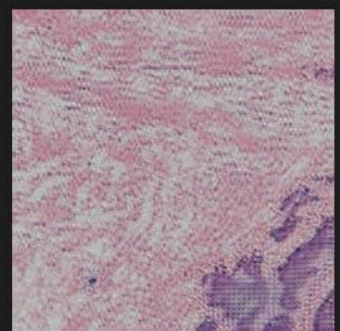
Source : IHC image(type_2)



Output from the model



Source : IHC image(type_3)



Output from the model

The outputs seem to be pixelated.

2. CycleGAN with modifications

I tried increasing the epochs to 100 (decay after 85), a much higher learning rate of 0.008, and a bigger batch size of 6 because I was under the impression that the model might be underfitting, and hence might not be learning the finer features(hence probably the pixellation).

I turned out to be terribly wrong, the learning rate was too high, especially for an unstable architecture based on GANs(this observation was obviously made in retrospect)

I tried to change the architecture to a UNet but it gave me the same results.

3. Residual UNet with Identity Loss

To improve the previous experiment, we thought of varying the identity loss and using a new architecture which was a UNet with Residual Blocks which used nearest neighbour upsampling instead of the standard deconvolutional layers in a normal UNet. I tried implementing the architecture mentioned in the [paper](#), but was unsuccessful despite multiple attempts to get the forward pass to work without breaking. There seemed to be a problem of the channels not matching during concatenating the skip connections. The code snippets of different permutations and combinations of the architecture(which goes by the name of a Residual UNet or ResUNet for short) is below :

```
import torch
import torch.nn as nn

class ResidualBlockForResUNetGenerator(nn.Module):
    def __init__(self, channels):
        super(ResidualBlockForResUNetGenerator, self).__init__()

        self.conv1 = nn.Conv2d(channels, channels, kernel_size=3, padding=1)
        self.norm1 = nn.InstanceNorm2d(channels)
        self.relu = nn.ReLU(inplace=True)

        self.conv2 = nn.Conv2d(channels, channels, kernel_size=3, padding=1)
        self.norm2 = nn.InstanceNorm2d(channels)

        def forward(self, x):
            residual = x
            out = self.conv1(x)
            out = self.norm1(out)
            out = self.relu(out)

            out = self.conv2(out)
            out = self.norm2(out)

            out += residual
```

```

        out = self.relu(out)
        return out

class
    ResUNetGenerator(nn.        output_channels,        base_channels=32):
        Module): # Modelled
        after a ResUNet
        def _init_(self,
            input_channels,
            super()._init_()

            self.encoder = nn.ModuleList()
            self.decoder = nn.ModuleList()

            # Encoder
            channels = base_channels
            for _ in range(3):
                self.encoder.append(nn.Conv2d(input_channels, channels,
kernel_size=4, stride=2, padding=1))
                self.encoder.append(nn.InstanceNorm2d(channels))
                self.encoder.append(nn.LeakyReLU(0.2,inplace=True))

            self.encoder.append(ResidualBlockForResUNetGenerator(output_channels))
                input_channels = channels
                channels *= 2

            # Middle
            self.middle = ResidualBlockForResUNetGenerator(channels)

            # Decoder
            for _ in range(3):
                self.decoder.insert(0, nn.Upsample(scale_factor=2, mode='nearest'))
                self.decoder.insert(0, nn.Conv2d(channels, input_channels,
kernel_size=3, padding=1))
                self.decoder.insert(0, nn.InstanceNorm2d(input_channels))
                self.decoder.insert(0, nn.LeakyReLU(0.2,inplace=True))
                self.decoder.insert(0,
ResidualBlockForResUNetGenerator(input_channels)) # Add a ResNet block
                    channels //= 2
                    input_channels = channels

            self.output = nn.Conv2d(input_channels, output_channels, kernel_size=7,
padding=3)
            self.tanh = nn.Tanh()

def forward(self, x):
    skip_connections = []

```

```

# Encoder
    for layer in self.encoder:
        x = layer(x)
skip_connections.append(x)

# Middle
x = self.middle(x)

# Decoder
    for layer in self.decoder:
        x = layer(x)
        if isinstance(layer, nn.Upsample):
            skip_connection = skip_connections.pop()
x = torch.cat((x, skip_connection), dim=1)

# Output
x = self.output(x) x =
    self.tanh(x)
    return x

# Example usage
input_channels,output_channels=3,3
generator_A_to_B = ResUNetGenerator(input_channels, output_channels)
generator_B_to_A = ResUNetGenerator(input_channels, output_channels)

print(generator_A_to_B)
print(generator B to A)

```

```

import torch
import torch.nn as nn

class ResidualBlock(nn.Module):
    def __init__(self, in_channels, out_channels):
        super(ResidualBlock, self).__init__()

        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3,
padding=1)
self.relu = nn.ReLU()
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
padding=1)
self.conv1x1=nn.Conv2d(out_channels,out_channels,kernel_size=1)

    def forward(self, x):

```



```

residual = x
print(f"Size of residual before 1x1 convolution : {residual.size()}")
    residual=self.conv1x1(residual)
print(f"Size of residual after 1x1 convolution : {residual.size()}") out =
    self.conv1(x)
print(f"After first conv layer : {out.size()}") out =
    self.relu(out)
out = self.conv2(out)
print(f"After second conv layer : {out.size()}")

# Adding the residual connection out +=
    residual

out = self.relu(out) return
    out

```

Second variation :

```

class ResidualBlockForResUNetGenerator2(nn.Module):
    def __init__(self, in_channels, out_channels):

        super(ResidualBlockForResUNetGenerator2, self).__init__() self.conv1
        = nn.Conv2d(in_channels, out_channels, kernel_size=3,
padding=1)
        self.conv2 = nn.Conv2d(out_channels, out_channels//2, kernel_size=3,
padding=1)

self.conv1x1=nn.Conv2d(out_channels//2,(out_channels//2)//2,kernel_size=1
) # self.conv3 = nn.Conv2d(out_channels//2, (out_channels//2)//2,
kernel_size=3, padding=1)
        # self.conv4 = nn.Conv2d((out_channels//2)//2,
((out_channels//2)//2)//2, kernel_size=3, padding=1)
        # self.conv5= nn.Conv2d(((out_channels//2)//2)//2, 3, kernel_size=3,
padding=1)
        self.norm = nn.InstanceNorm2d(out_channels)
        self.activation = nn.LeakyReLU(0.2)

    def forward(self, x):
        residual = x
        print(f"Size of residual/original input : {residual.size()}")
        residual=self.conv1x1(residual)
        print(f"Size of residual/input tensor : {residual.size()}")
        x = self.conv1(x)
        print(f"dimensions of x after first conv layer in residual block :
{x.size()}")

```

```

        x = self.norm(x)
        x = self.activation(x)
        x = self.conv2(x)
        print(f"dimensions of x after second conv layer in residual block :
{x.size()}")
        x = self.norm(x)
        # x = self.conv3(x)
        # print(f"dimensions of x after third conv layer in residual block :
{x.size()}")
        # x = self.norm(x)
        # x = self.conv4(x)
        # print(f"dimensions of x after fourth conv layer in residual block :
{x.size()}")
        # x = self.norm(x)
        # x = self.conv5(x)
        # print(f"dimensions of x after fifth conv layer in residual block :
{x.size()}")
        # x = self.norm(x)
        x += residual
        x = self.activation(x)
        return x

class ResUNetGenerator2(nn.Module,
                        output_channels=128,
                        base_channels=32,
                        input_channels=1,
                        depth=3):

    def __init__(self,
                  output_channels=128,
                  base_channels=32,
                  input_channels=1,
                  depth=3):
        super(ResUNetGenerator2, self).__init__()

        # Encoder
        self.encoder_layers = nn.ModuleList()
        in_channels = input_channels
        out_channels = base_channels
        for _ in range(depth):

            self.encoder_layers.append(ResidualBlockForResUNetGenerator2(in_channels,
                                                                           out_channels))
            self.encoder_layers.append(nn.MaxPool2d(kernel_size=2, stride=2))
            in_channels = out_channels
            out_channels*=2

        # print(f"After encoder, before decoder : The in_channels =
        {in_channels}, out_channels = {out_channels}")

        # in_channels : 64, out_channels: 128

        # Middle block

```

```

        # self.middle = ResidualBlockForResUNetGenerator2(in_channels,
in_channels)

        # Decoder
        self.decoder_layers = nn.ModuleList()
        for _ in reversed(range(depth)):
            self.decoder_layers.append(nn.Upsample(scale_factor=2,
mode='nearest'))

self.decoder_layers.append(ResidualBlockForResUNetGenerator2(in_channels +
out_channels, out_channels))
            out_channels//=2
            in_channels = out_channels

        # print(f"After decoder, before output layer : The in_channels =
{in_channels}, out_channels = {out_channels}")
        # in_channels : 16 out_channels : 16

        # Output layer
        self.output_conv = nn.Conv2d(out_channels, output_channels,
kernel_size=3, padding=1)

def forward(self, x):
    # skip_connections = []
    # for layer in self.encoder_layers:
    #     x = layer(x)
    #     skip_connections.append(x)

    # # x = self.middle(x)

    # for layer, skip in zip(self.decoder_layers[::-2],
reversed(skip_connections)):
        #     x = layer(x)
        #     x = torch.cat([x, skip], dim=1)
        # x = self.output_conv(x)
        # return x

    skip_connections = []

    # Encoder
    for layer in self.encoder_layers:
        x = layer(x)
        skip_connections.append(x)

```

```

# Middle
# x = self.middle(x)

# Decoder
    for layer in self.decoder_layers:
        x = layer(x)
        if isinstance(layer, nn.Upsample):
            skip_connection = skip_connections.pop()
x = torch.cat((x, skip_connection), dim=1)

# Output
x = self.output(x) # x =
    self.tanh(x)
    return x

```

```

class ResidualBlockForResUNetGenerator2(nn.Module):
    def __init__(self, in_channels, out_channels):

        super(ResidualBlockForResUNetGenerator2, self).__init__() self.conv1
        = nn.Conv2d(in_channels, out_channels, kernel_size=3,
padding=1)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
padding=1)
        self.conv1x1 = nn.Conv2d(in_channels, out_channels , kernel_size=1) #
1x1 convolution for shortcut connection
        self.norm = nn.InstanceNorm2d(out_channels)
        self.activation = nn.LeakyReLU(0.2)
        self.out_channels = out_channels # Store out_channels as an instance
variable

    def forward(self, x):
        residual = x
        print(" ")
        print(f"Size of residual/original input : {residual.size()}")

        shortcut_channels = self.out_channels // 2 # Determine
shortcut channel count dynamically
        print("Shortcut channels:", shortcut_channels)

        if residual.size(1) != shortcut_channels:
            # pad_channels = shortcut_channels - residual.size(1)
            # print("Pad channels:", pad_channels)
            # padding_left = pad_channels // 2

```

```

        # padding_right = pad_channels - padding_left
        residual = self.conv1x1(residual) # Apply the conv1x1 layer to
match channels
        print(f"Size of residual/input tensor : {residual.size()}")
        # residual = F.pad(residual, (padding_left, padding_right, 0, 0))
# Adjust padding dimensions
        # print("Padded residual shape:", residual.size())

x = self.conv1(x)
print("After conv1 shape:", x.size())

x = self.norm(x)
print("After norm1 shape:", x.size())

x = self.activation(x)
print("After activation1 shape:", x.size())

x = self.conv2(x)
print("After conv2 shape:", x.size())

x = self.norm(x)
print("After norm2 shape:", x.size())

# Resize the residual tensor to match the number of channels of x
# residual = F.interpolate(residual, size=x.size()[2:], mode='nearest')
# residual = residual[:, :x.size(1), :, :] # Match the number of
channels
# print("Resized residual shape:", residual.size())

x += residual
x = self.activation(x)
print("Output shape:", x.size())

return x

# Create a sample input tensor
input_channels = 32
output_channels = 64
batch_size = 6
input_size = (512, 512)

input_tensor = torch.randn(batch_size, input_channels, *input_size)

# Create an instance of the ResidualBlockForResUNetGenerator2 class
residual_block = ResidualBlockForResUNetGenerator2(input_channels,

```

```

output_channels)
#residual_block = ResidualBlockForResUNetGenerator2(input_channels,
input_channels) # Or any other desired output channels

# Pass the input tensor through the residual block
output_tensor = residual_block(input_tensor)

# Print the shapes of input and output tensors
print("Input tensor shape:", input_tensor.shape)
print("Output tensor shape:", output_tensor.shape)

```

Third variation :

```

class ResUNetGeneratorTest(nn.Module):
    def __init__(self, input_channels, output_channels, base_channels=32,
depth=3):
super(ResUNetGeneratorTest, self).__init__()

# Encoder
self.encoder_layers = nn.ModuleList()
    in_channels = input_channels
    out_channels = base_channels
for _ in range(depth):

self.encoder_layers.append(ResidualBlockForResUNetGeneratorTest(in_channels,
out_channels))
self.encoder_layers.append(nn.MaxPool2d(kernel_size=2, stride=2)) in_channels =
out_channels
    if not depth-__==1:
        out_channels*=2

print(f"After encoder, before decoder : The in_channels =
{in_channels}, out_channels = {out_channels}")
    # in_channels : 128, out_channels: 128

#[6,128,64,64]
# Middle block

    #possibly loosing some feature info by not expanding the channels while
reducing spatial dimensions here :
    self.middle =
nn.Sequential(ResidualBlockForResUNetGeneratorTest(in_channels, out_channels),
nn.MaxPool2d(kernel_size=2, stride=2))

```

```

        # out_channels//=2

        #[6,128,32,32]
        # Decoder
        self.decoder_layers = nn.ModuleList()
        for _ in range(depth):

            self.decoder_layers.append(nn.Upsample(scale_factor=2,
mode='nearest'))#nearest neighbour upsampling

self.decoder_layers.append(ResidualBlockForResUNetGeneratorTest(out_channels
, out_channels//2))#out_channels here changes every decoder block
        # Input to first decoder residual block :
        6,256,64,64 # Input to second decoder res block : 6,
        192, 128,128 # Input to third decoder res block :
        6,128,256,256
        # Output of the third decoder block :
        6,64,256,256 if not depth-==1:
            out_channels//=2
            # in_channels = out_channels

        # print(f"After decoder, before output layer : The in_channels =
        {in_channels}, out_channels = {out_channels}")
        # in_channels : 32 out_channels : 32

        # Output layer
        self.output=nn.Sequential(
            nn.Upsample(scale_factor=2,mode="nearest"),
            nn.Conv2d(out_channels, output_channels, kernel_size=1, padding=1)
        )
        # self.output_conv = nn.Conv2d(out_channels, output_channels,
        kernel_size=1, padding=1)#output_channels=number of channels you want the
        generator's output to have

def forward(self, x):

    skip_connections = []
    # Encoder
    print("----- Encoder starts")
    for layer in self.encoder_layers:
        x = layer(x)
        if isinstance(layer,nn.MaxPool2d):
            skip_connections.append(x)

```

```

print("----- Middle starts      ")
# Middle
x = self.middle(x)
print(" ")
print("Tensors to be concatenated to x : ")

        for sc in skip_connections:
            print(sc.size())
# Decoder
print("----- Decoder starts      ")
for layer in self.decoder_layers:

        if isinstance(layer, nn.Upsample):
            x=layer(x)
skip_connection = skip_connections.pop()
            print(f"Concatenating skip connection :
{skip_connection.size()} to x : {x.size()}")
            x = torch.cat((x, skip_connection), dim=1)
        else:
x = layer(x) # Upsampling, skip connection, residual block

# Output
x = self.output(x) # x =
        self.tanh(x)
        return x

```

```

class ResidualBlockForResUNetGeneratorTest(nn.Module):
    def __init__(self, in_channels, out_channels):
super(ResidualBlockForResUNetGeneratorTest, self).__init__() self.downsample = None
        if in_channels == out_channels else
nn.Conv2d(in_channels, out_channels, kernel_size=1)
        self.conv1 = nn.Conv2d(in_channels, out_channels, kernel_size=3,
padding=1)
        self.conv2 = nn.Conv2d(out_channels, out_channels, kernel_size=3,
padding=1)
self.norm = nn.InstanceNorm2d(out_channels)
        self.activation = nn.LeakyReLU(0.2)
        self.out_channels = out_channels

def reduce_channels(self, x, target_channels):

```



```

        # Create a 1x1 convolutional layer to adjust the number of channels
        conv = nn.Conv2d(x.size(1), target_channels, kernel_size=1)
        return conv(x)

    def forward(self, x):
        residual = x
        x = self.conv1(x)
        x = self.norm(x)
        x = self.activation(x)
        x = self.conv2(x)
        x = self.norm(x)
        x = self.activation(x)

        if self.downsample:
            residual = self.downsample(residual)

        if residual.shape[1] != x.shape[1]:
            residual = self.reduce_channels(residual, x.shape[1])

        x += residual
        x = self.activation(x)

        return x

class ResUNetGeneratorTest(nn.Module):
    def __init__(self, input_channels, output_channels, base_channels=32, depth=3):
        super(ResUNetGeneratorTest, self).__init__()

        self.encoder_layers = nn.ModuleList()
        in_channels = input_channels
        out_channels = base_channels

        for _ in range(depth):
            self.encoder_layers.append(ResidualBlockForResUNetGeneratorTest(in_channels, out_channels))
            self.encoder_layers.append(nn.MaxPool2d(kernel_size=2, stride=2))
            in_channels = out_channels
            if not depth - _ == 1:
                out_channels *= 2

        self.middle = nn.Sequential(ResidualBlockForResUNetGeneratorTest(in_channels, out_channels),
                                     nn.MaxPool2d(kernel_size=2, stride=2))

```

```

        self.decoder_layers = nn.ModuleList()
        in_channels_decoder = out_channels

        for _ in range(depth):

self.decoder_layers.append(ResidualBlockForResUNetGeneratorTest(in_channels_de
c_oder, in_channels_decoder // 2))
            self.decoder_layers.append(nn.Upsample(scale_factor=2,
mode='nearest'))

        self.output = nn.Sequential(
            nn.Upsample(scale_factor=2, mode="nearest"),
            nn.Conv2d(in_channels_decoder, output_channels, kernel_size=1,
padding=0)
        )

    def reduce_channels(self, x, target_channels):
        # Create a 1x1 convolutional layer to adjust the number of channels
        conv = nn.Conv2d(x.size(1), target_channels, kernel_size=1)
        return conv(x)

    def forward(self, x):
        skip_connections = []

        # Encoder
        print("----- Encoder starts")
        for layer in self.encoder_layers:
            x = layer(x)
            if isinstance(layer, nn.MaxPool2d):
                skip_connections.append(x)

        x = self.middle(x)

        # Initialize in_channels_decoder here
        in_channels_decoder = self.decoder_layers[0].out_channels#

        # Decoder
        print("----- Decoder starts")
        # count=0
        for layer in self.decoder_layers:
            # print('number of times decoder called: ', count)
            if isinstance(layer, nn.Upsample):

```

```

        if len(skip_connections) > 0:
            skip_connection =
            skip_connections.pop()
            print(f"Before upsampling, x size: {x.size()}, OG
skip_connection size: {skip_connection.size()}")
            # Adjust the number of channels in skip_connection
            skip_connection = self.reduce_channels(skip_connection,
in_channels_decoder)
            print(f"skip_connection size after 1x1 conv :
{skip_connection.size()}")
            # Ensure that skip_connection has the same spatial
dimensions as x
            skip_connection = F.interpolate(skip_connection,
size=x.shape[2:], mode='nearest')
            print(f"skip_connection after interpolation :
{skip_connection.size()}")
            x = torch.cat((x, skip_connection), dim=1)
            in_channels_decoder //= 2
            print(f"After concatenating x and skip_connection, x :
{x.size()}")
            x = layer(x)
            print(f"After
upsampling x :
{x.size()}")
        else:
            print(f"x
before
Residual
Block:
{x.size()}")
            x = layer(x)
            print(f"x
after Residual
Block :
{x.size()}") #
            count += 1

# Output
x = self.output(x)
return x

```

```

modelTest = ResUNetGeneratorTest(3, 3)
# Define a list of image dimensions to test
image_dimensions = [(6, 3, 512, 512)]
for image_dim in image_dimensions:
    image = torch.randn(*image_dim)
    output = modelTest(image)
    print(f"Input dimension: {image_dim}, Output dimension: {output.size()}")

```

4. StainGAN We switched to a [stainGAN](#) architecture which uses a ResNet generator and a 70x70 PatchGAN discriminator and trained it on type_0 images, a batch_size of 4, with Instance normalization, and for 100 epochs, with the learning rate decaying after 60 epochs.

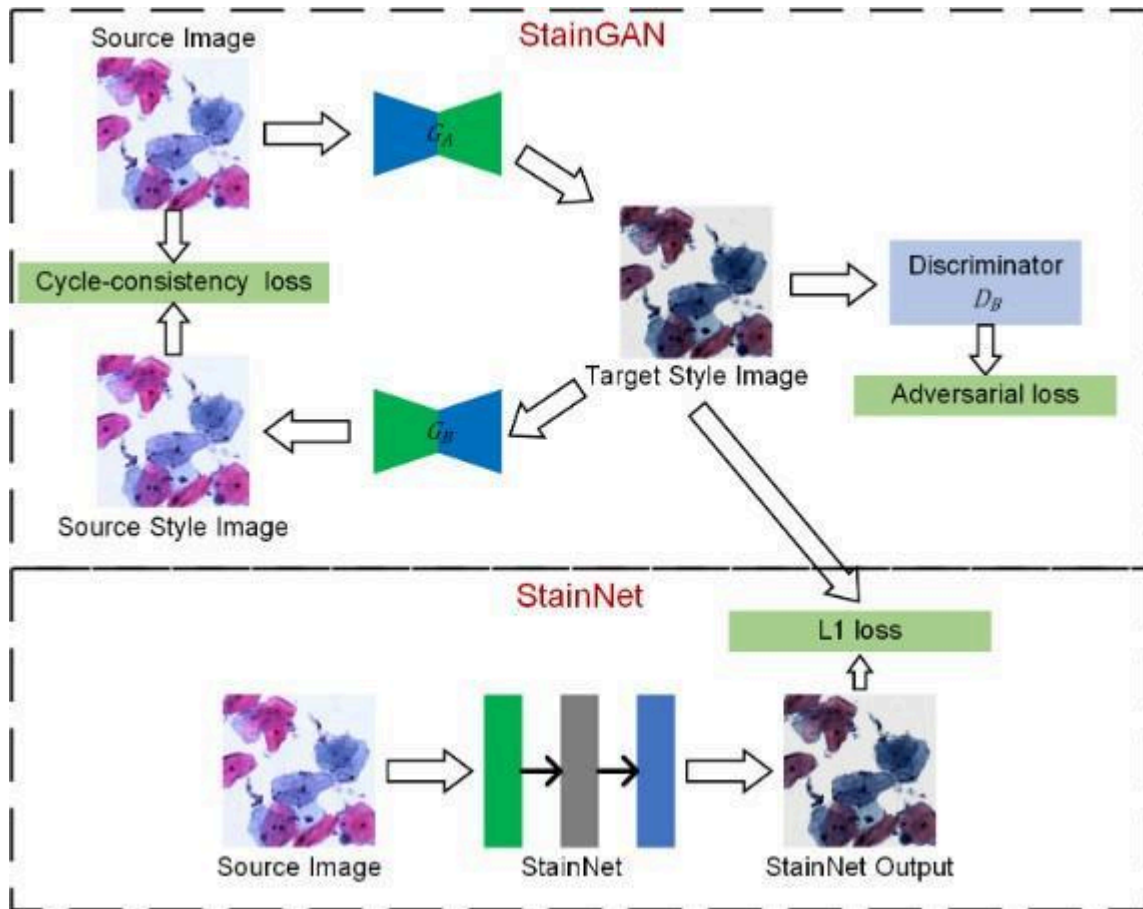


Source : IHC image(type_0)

Output from stainGAN

5. StainNet

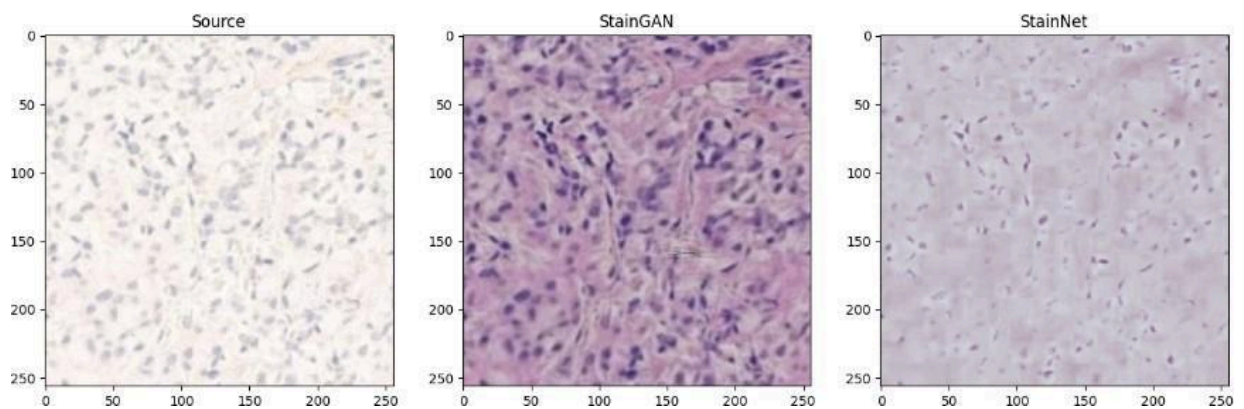
Although stainGAN's output seemed promising, we thought we could improve it further through [stainNet](#) which used a 1x1 convolutional network on the output of stainGAN. This [model](#) would treat the output of StainGAN as ground truth, and train the fully 1x1 conv network with an L1 Loss using an SGD optimiser.



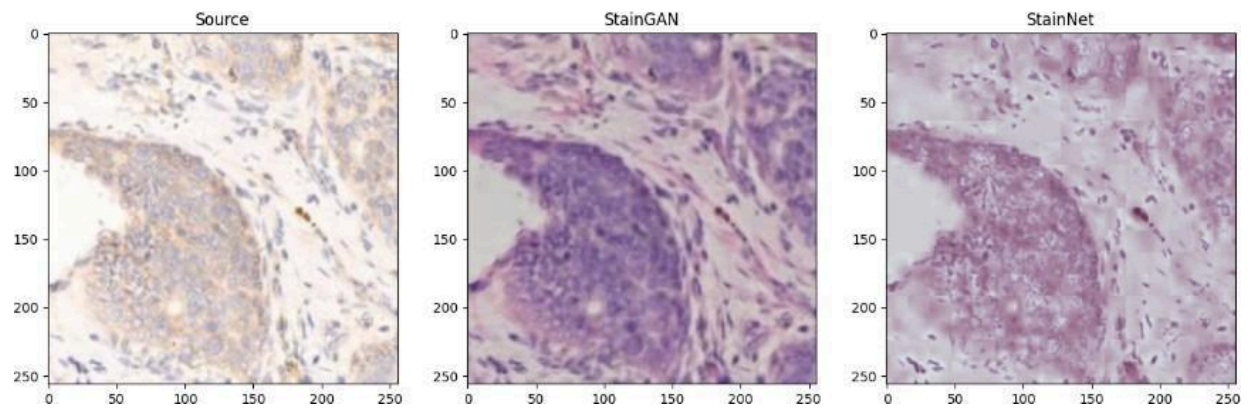
From the [paper](#) above

These are some results on the different types of BCI images :

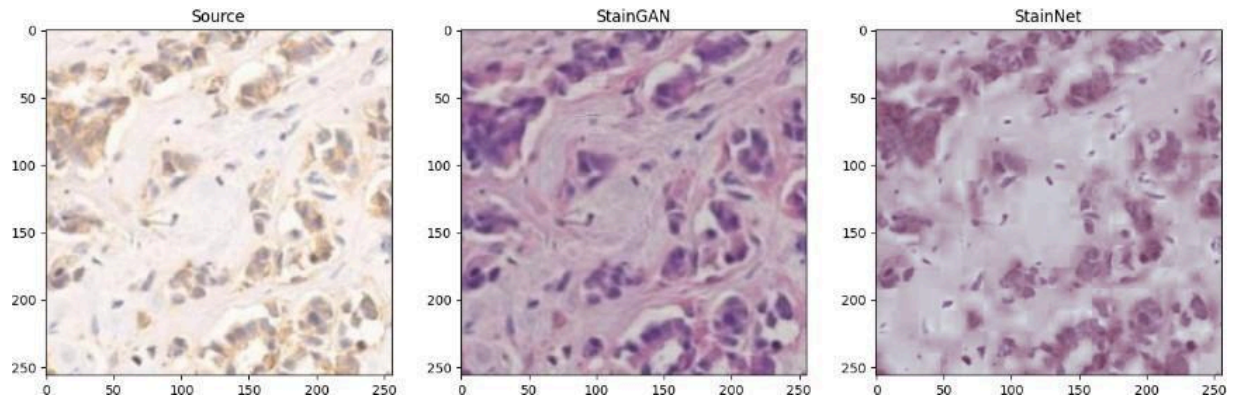
type_0:



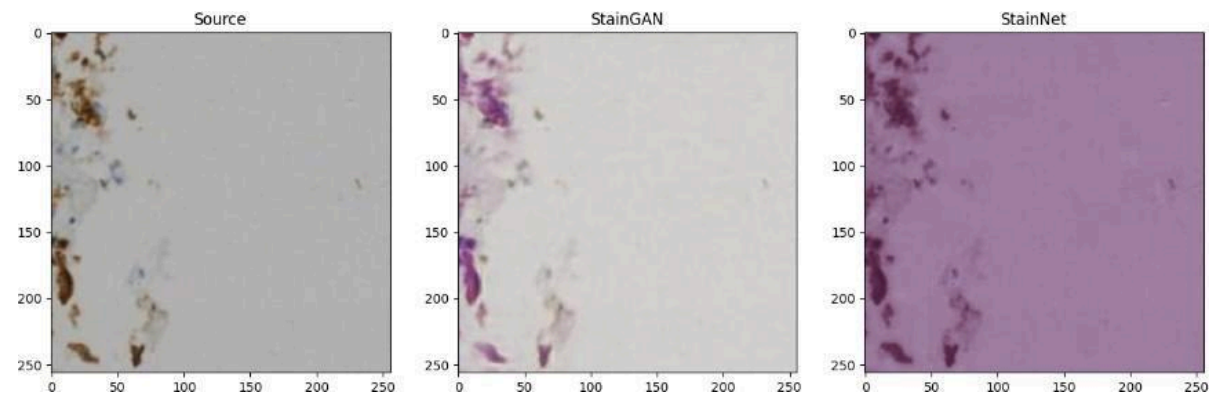
type_1:



type_2:

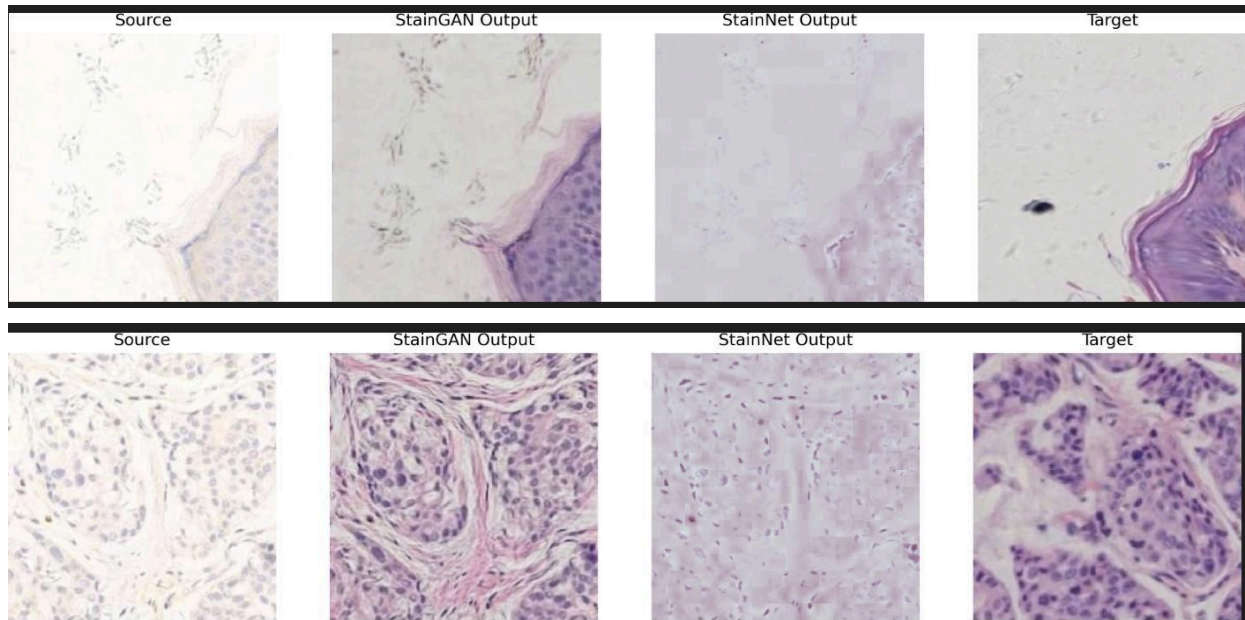


type_3:

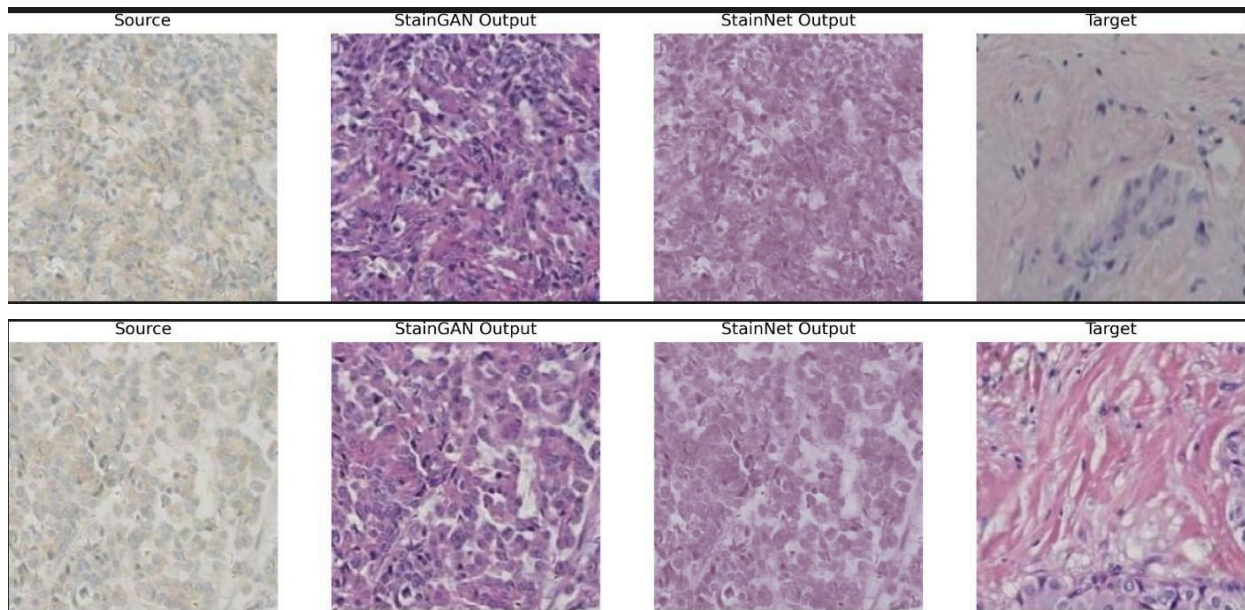


Here are some results with ground truth images(what the slice actually looks like in the target stain):

type_0:

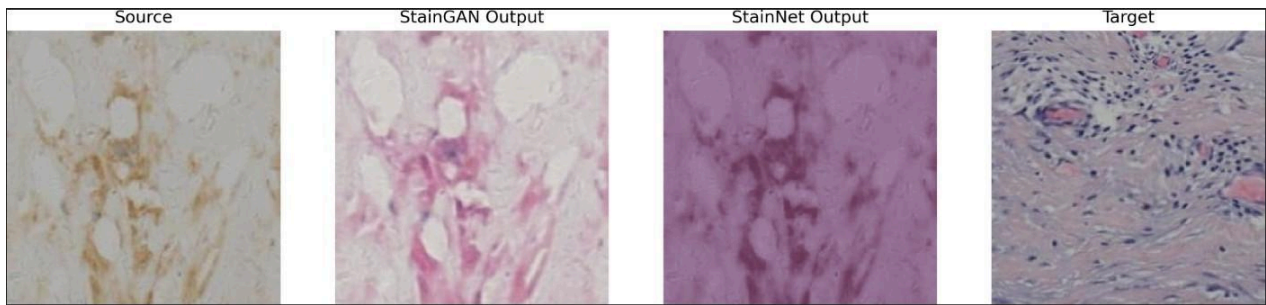
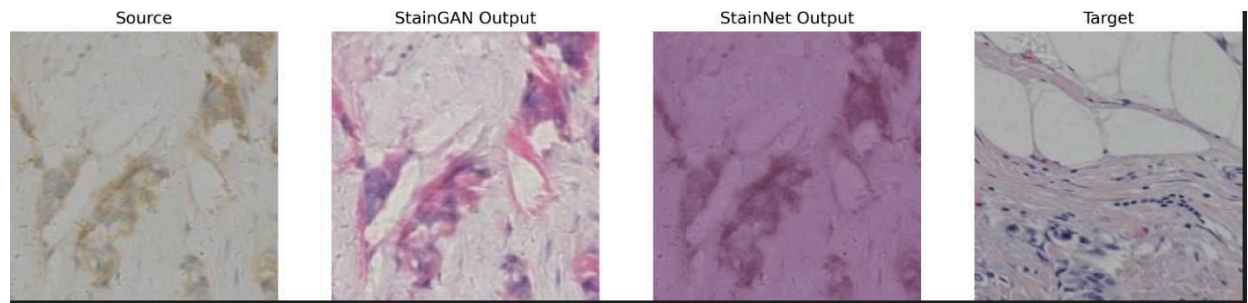


type_1:

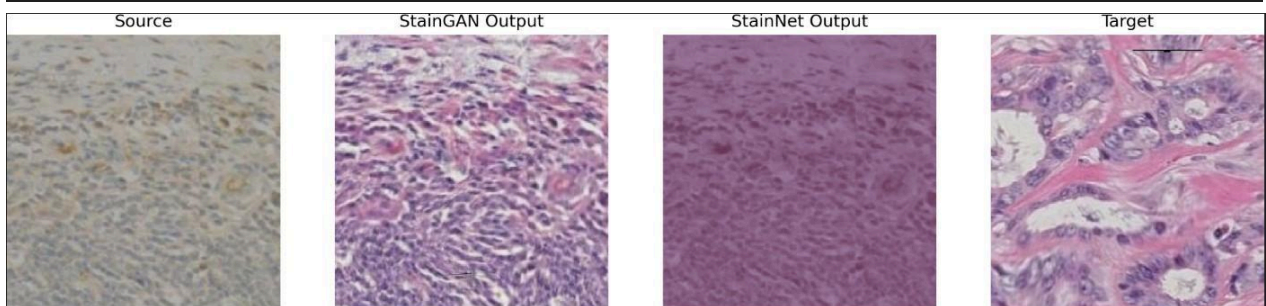
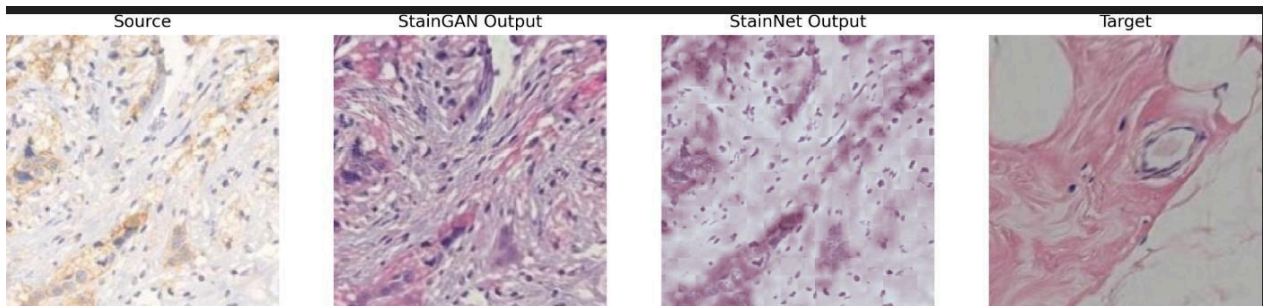


This is when we started to realise that some of the target images probably weren't of the same slice. So the outputs of the models looked closer in overall structure than the "ground truth" itself.

type_2:



type_3:



6. StainGAN on the allen dataset

We decided to go with StainGAN as it visually outperformed stainNet. We switched to the allen dataset (21pcw Cerebrum slices). The images we downloaded were of different resolutions, so I cropped them to the smallest combination of height x width available in the data, which came out to be 1508x880 pixels.

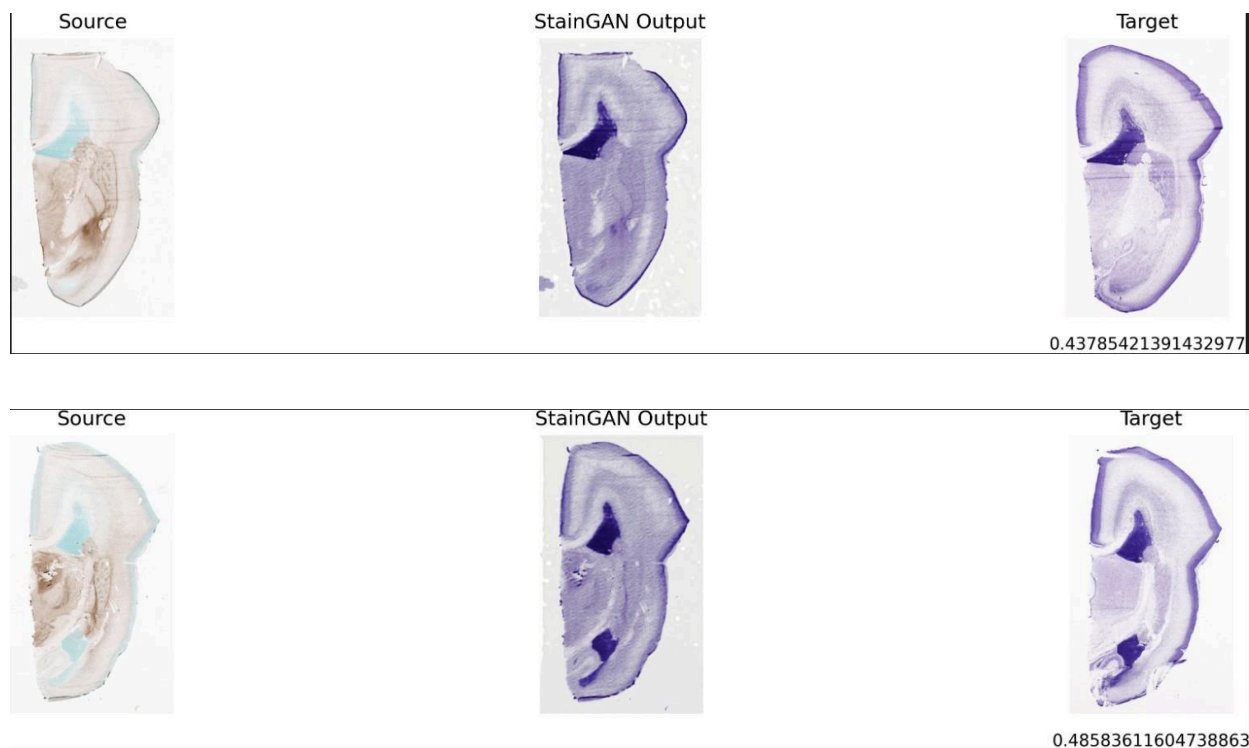
Around this time, I also found out in the issues section of the [original cycleGAN github repository](#)

That the model can take non-patched (cropped) images as well(it would crop the images automatically). That would serve us well, as we wouldn't need to find a way to stitch the patches together after inference without the overlap(this is something I struggled with for weeks).

We trained a ResNet generator with 9 blocks, the standard PatchGAN discriminator, a learning rate of 0.0002, a batch size of 4, instance normalization, 100 epochs(decay after 60) on the whole resized images from the allen dataset.

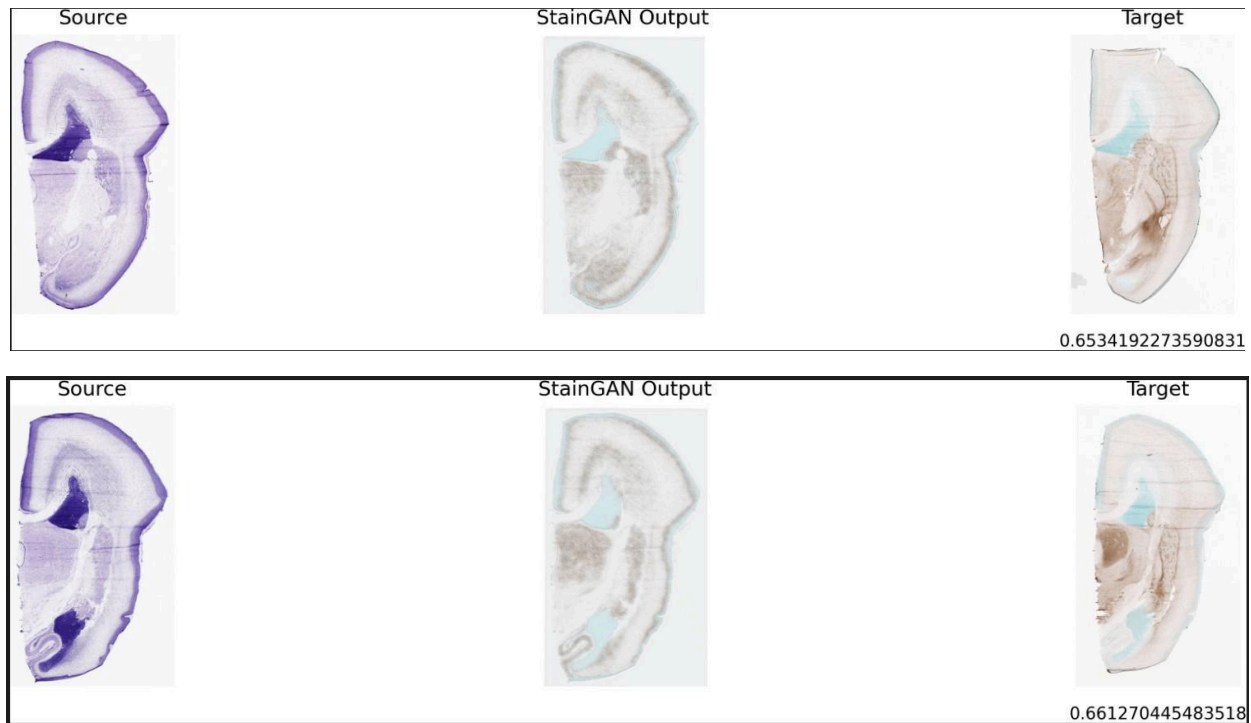
Here are some of the intermediate results during training :

From an ACHE stain to the NISSL stain:



The decimal number on the bottom right is the SSIM score(measure of structural similarity between two images) of the output and the target. It's measured on a scale of -1 to 1, -1 for absolute dissimilarity and 1 for absolute similarity. These SSIM scores are low for these slices because the target isn't from the same slice, there are slight differences between the source (and by extension the model's output slice) slice and the target slice. They were likely sliced one after the other at the same area, which is why we chose to compare them in the first place.

From the NISSL stain to ACHE stain:



7. StainGAN with self attention and spectral normalization

We deduced that the model likely requires an attention layer to be able to capture more features. I came upon a [paper](#) from a student at Stanford that used self attention in a cycle GAN along with spectral normalization on translating pictures of faces to their manga(Japanese comic style) equivalent.

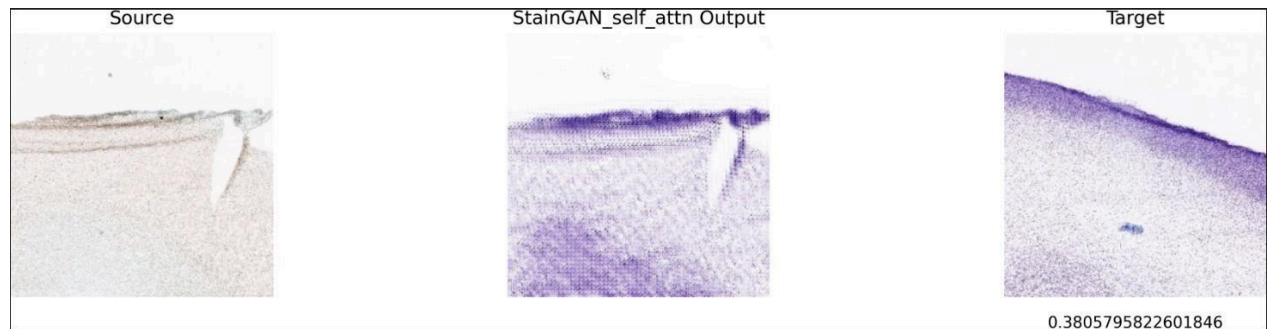
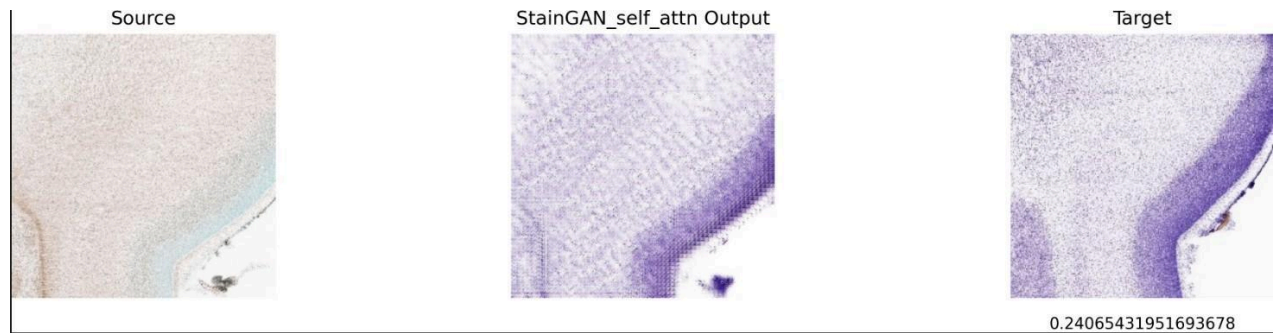
The architecture in the paper used a ResNet with 9 residual blocks, and a PatchGAN, with a self attention after layer 2 and layer 3 in the Generators, and after layer 3 and before the output layer (Conv layer followed by a sigmoid layer to output a single number) in the Discriminators . Both the Generators and the Discriminators used spectral normalization(which according to the paper was supposed to help the model output images of a richer quality).

We trained the model just like the previous ones, with a higher learning rate, but the model seemed to not learn any features and just returned full color images. Unfortunately I couldn't experiment much with the model, like using spectral norm only in the Generator or only in the Discriminator or trying a lower learning rate because I ended up using instance normalization and self attention and started to get promising results.

8. StainGAN with self attention and instance normalization

I used self attention in both the Generators and the Discriminators, a batch size of 2, 100 epochs(decay after 50), instance normalization, and a learning rate of 0.0002. The results that

came out were better than anything we had seen until then, but still looked blurry and not of the best resolution.



We decided to increase the number of attention layers, but ran into memory problems because the feature size (spatial resolution) was too large for the attention operation in the layers before downsampling and after upsampling of the image. It was also around this time that during inference, while passing in the whole image we started to get memory problems, because the attention layers in the generators were used to 256x256 patches, and we were giving them 1508x880 size whole images.

Cropping logic :

```
width, height = 1024, 1024

# Define the size of the patches
patch_size = 256
image_patches=[]
c=0
patch_coordinates={}
# Iterate through the image and extract 256x256 patches
for x in range(0, width, patch_size):
for y in range(0, height, patch_size):
# Define the coordinates for cropping
```

```

left = x upper =
    y
right = x + patch_size lower =
    y + patch_size c+=1
# print(f"Left,Upper : {left},{upper}")
# print(f"Right, Lower : {right},{lower}") #
    print(f"Location of patch : {c}")
patch_coordinates[c]=((left,upper),(right,lower))

print(patch_coordinates)
pasting_coordinates={}
for k,v in patch_coordinates.items():
    pasting_coordinates[k]=v[0]

print(pasting_coordinates)

```

Stitching logic:

```

def stitch_patches(patches_paths,original_width,original_height):
    """
        patches : a list of (16) patches(paths to the patches) from source_paths
        list, all representing one image
    """
    reconstructed_image = Image.new("RGB", (original_width, original_height))
    for i,path in enumerate(patches_paths):
        img_num=path[18:23]# image number
        stainGAN_patches=[p for p in stainGAN_outputs[:len(patches_paths)]]
        patch=Image.fromarray(stainGAN_patches[i])
        patch_num=int(path[38:].rstrip(".png"))# patch number in source_paths
        list
        print(patch_num) reconstructed_image.paste(patch,
        (pasting_coordinates[patch_num][0],pasting_coordinates[patch_num][1]))

    reconstructed_image.save(f"./reconstructed_original_image_{img_num}.png")

```

The outputs of the models, when given the BCI dataset patches as input during inference, still showed the areas of overlap. This would be a problem in real life, as the stitched image that's output by the model cannot have aberrations.

I tried gaussian blurring the image, using binary thresholding(on the grayscale version of the original image) to create a mask(pixel values greater than 60 are turned white, and the others

become black), finding and drawing the contours on the mask, and outputs an combination of the original image blended with the blurred image based on the mask.

```
image = cv2.imread('./reconstructed_original_image_00292.png')
blurred_img = cv2.GaussianBlur(image, (21, 21), 0)
mask = np.zeros(image.shape, np.uint8)

gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
(rv,thresh) = cv2.threshold(gray, 60, 255, cv2.THRESH_BINARY)
contours, hierarchy = cv2.findContours(thresh.copy(), cv2.RETR_EXTERNAL,
cv2.CHAIN_APPROX_SIMPLE)

cv2.drawContours(mask, contours, -1, (255,255,255),5)
output = np.where(mask==np.array([255, 255, 255]), blurred_img, image)
```

But the image just looked slightly less bright, with the overlapping still

visible. I tried alpha blending :

```
def
stitch_patches_alpha_blending(patches_paths,original_width,original_height,sta
r t_index,end_index):
    """
    patches : a list of (16) patches(paths to the patches) from source_paths
list, all representing one image
    """
    reconstructed_image = Image.new("RGB", (original_width, original_height))
    for i,path in enumerate(patches_paths[start_index:end_index]):
        img_num=path[18:23]# image number
        patch_num=int(path[38:].rstrip(".png"))# patch number in source_paths
list
        stainGAN_patches=[p for p in stainGAN_outputs[start_index:end_index]]
        patch=Image.fromarray(stainGAN_patches[i])

        target_patch=reconstructed_image.crop((patch_coordinates[patch_num][0][0],patch
_coordinates[patch_num][0][1],patch_coordinates[patch_num][1][0],patch_coordina
tes[patch_num][1][1]))
        result = ImageChops.blend(target_patch, patch, 0.80)
        reconstructed_image.paste(result,
(pasting_coordinates[patch_num][0],pasting_coordinates[patch_num][1]))

    reconstructed_image.save(f"./reconstructed_original_image_alpha_blend_{img_num}
.png")
```

This code essentially gives a alpha(transparency) value to each pixel in the stitched image that was output from the model and overlaps it onto the randomly initialized image, in order to smooth out the transition from pixel to pixel(overlaps usually mean a large pixel value difference in adjacent pixels)

But the resulting image again, just looked like a less bright version of the original stitched image, just with overlaps not glaringly obvious.

I also tried seamless cloning:

```
def stitch_patches_opencv(patches_paths,original_width,original_height):
    """
    patches : a list of (16) patches(paths to the patches) from source_paths
    list, all representing one image
    """

    reconstructed_image = Image.new("RGB", (original_width, original_height))
    reconstructed_image_array = cv2.cvtColor(np.array(reconstructed_image),
cv2.COLOR_RGB2BGR)
    print(f"Reconstructed Image Array : {reconstructed_image_array}")
    for i,path in enumerate(patches_paths):
        img_num=path[18:23]# image number
        stainGAN_patches=[p for p in stainGAN_outputs[:len(patches_paths)]]
        patch=stainGAN_patches[i]
        patch_num=int(path[38:].rstrip(".png"))# patch number in source_paths
list
        # Define the center point (x, y) where you want to paste the source
image onto the destination
        center_x, center_y = pasting_coordinates[patch_num][0]+(256//2) ,
pasting_coordinates[patch_num][1]+(256//2)
        # print(f"Image patch : {patch}")
        roi = reconstructed_image_array[center_y - patch.shape[0] // 2:
center_y + patch.shape[0] // 2,
                                center_x - patch.shape[1] // 2: center_x +
patch.shape[1] // 2]
        # print(f"ROI : {roi}")

        result = cv2.seamlessClone(patch, reconstructed_image_array, roi,
(center_x - roi.shape[1] // 2, center_y - roi.shape[0] // 2),
cv2.NORMAL_CLONE)
        # print(f"Result array : {result}")
        # cv2.imshow("Resulting Image", result)
        # cv2.imwrite(f"seamless_cloning_result{i}.png", result)
        # reconstructed_image.paste(patch,
```

```
(pasting_coordinates[patch_num][0],pasting_coordinates[patch_num][1]))
# reconstructed_image.save(f"./reconstructed_original_image_{img_num}.png") #
    Save the reconstructed original image
# cv2.imwrite("reconstructed_original_image_opencv.png", result)
```

Seamless cloning seeks to minimize the gradients between the selected region of the source image, and the target region in the target image.

No improvements seemed to occur.

I eventually tried a sliding window approach :

```
import numpy as np
image=Image.open(r"../allen_dataset_ache_nissl/testA/104_101892588_primary.jpg"
)
# Load your large image as a NumPy array (replace 'large_image' with your
array)
large_image = np.array(image)

# Define the sliding window parameters
window_height, window_width = 128, 128      # Adjust the size as
needed stride = 8 # Adjust the stride for overlapping windows

# Initialize an empty result array with a floating-point data type
output = np.zeros_like(large_image)

# Iterate through the image using sliding windows
for top in range(0, large_image.shape[0] - window_height, stride):
    for left in range(0, large_image.shape[1] - window_width, stride):
        # Extract the current window
        window = large_image[top:top + window_height, left:left +
window_width,:]

# Perform any necessary preprocessing (e.g., normalization)

# Make predictions using the model (replace with your inference code)
```



```

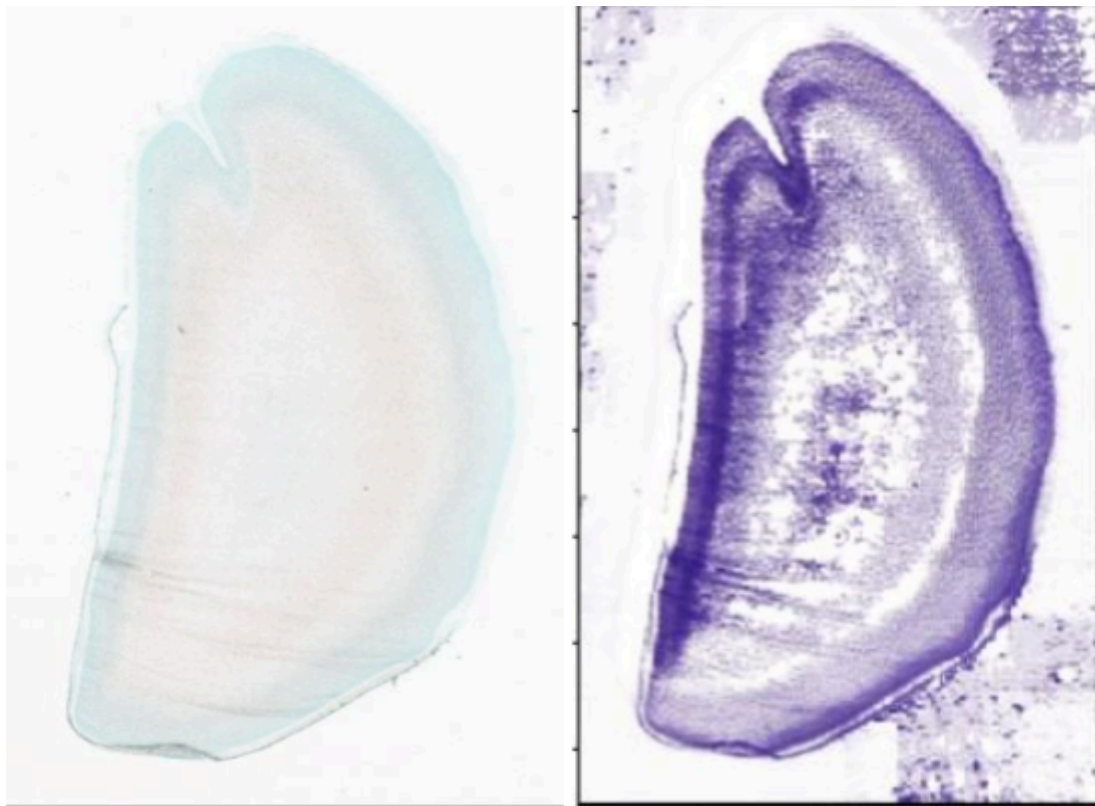
predictions = stainGANEval(model_GAN_self_attn_A_to_B>window)

# Place the predictions in the corresponding location of the output
array
    output[top:top + window_height, left:left + window_width,:] =
    predictions

# Normalize the output by the number of contributions (in the overlapping
regions)
num_contributions = np.zeros_like(large_image, dtype=np.float64)
for top in range(0, large_image.shape[0] - window_height, stride):
    for left in range(0, large_image.shape[1] - window_width, stride):
        num_contributions[top:top + window_height, left:left + window_width] +=
1

# output /= num_contributions
plt.imshow(output)

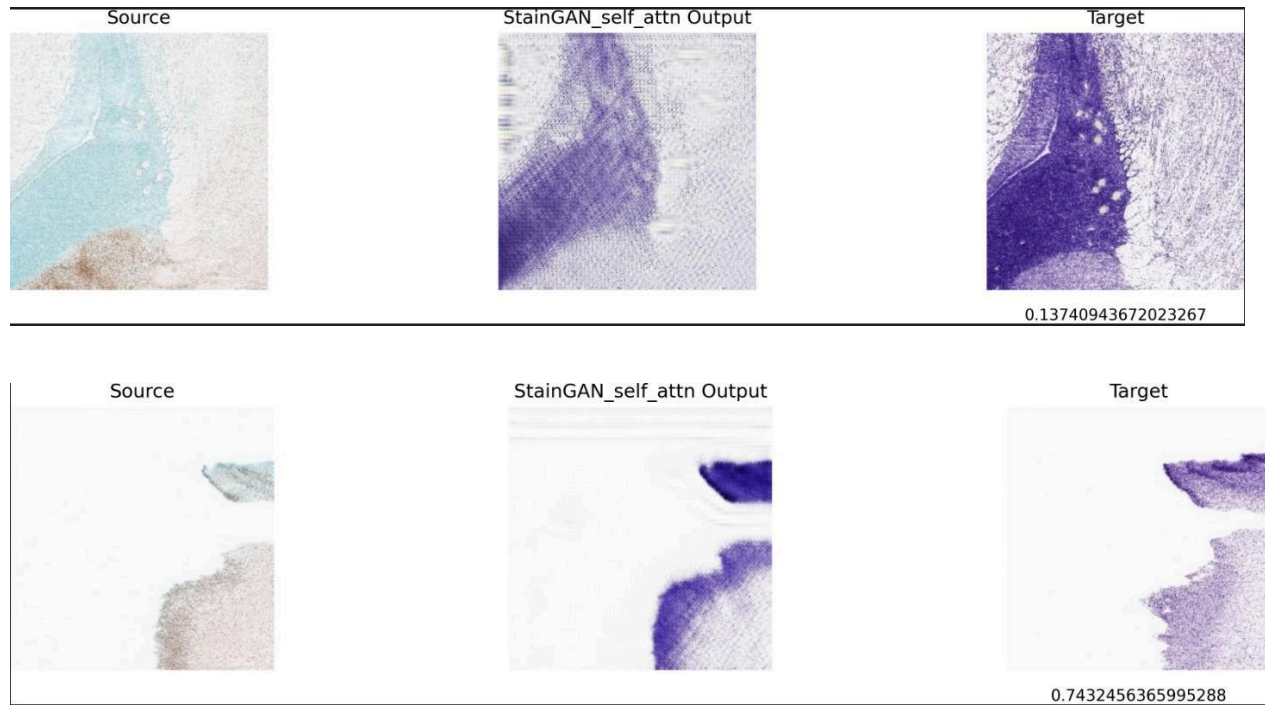
```



This approach was problematic because just inference on the whole image took over a minute to complete.

9. StainGAN + self attention only in the third layer of the Generator

The outputs for this model were also unsatisfactory.



10. StainGAN with 2 self attention layers along with augmentation and normalization

I came across a [paper](#) that combined color augmentation(a popular way to regularize training by increasing diversity of images in the dataset), and color normalization. It's ideal use case would be any problem with a slightly - very diverse dataset. The allen dataset is unfortunately not that diverse (it works slightly better for the BCI dataset because each type has 4 subtypes, each of which are different in structure).

This approach came about because of the need for doing better than the current augmentation techniques out there that do not produce realistic augmented images for medical images.

This augmentation+normalization was name RandStainNA :

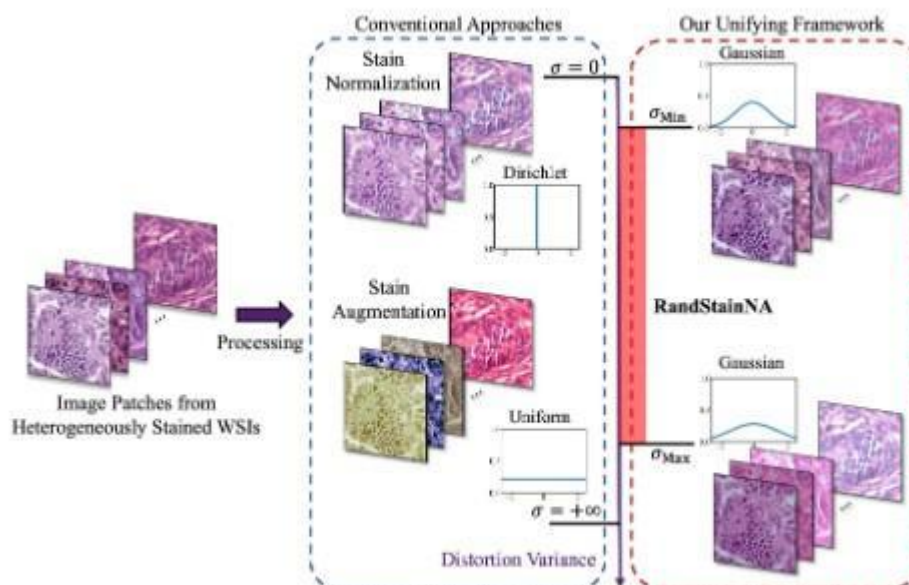


Fig. 1. The overall framework of the proposed RandStainNA.

I divided the dataset into ache and nissl images, under each stain, I used two images of “style1”. In a more diverse dataset, there would be multiple styles. Then the [code](#) generated a .yaml file with information about how to convert the images from the RGB space to a LAB space with configurations for the underlying distributions for the mean and standard deviation for the Lightness, Green-Red spectrum, and the Blue-Yellow spectrum, the three components that make up LAB space.

The configuration file for the ache images looked like this:

```
A:
avg:
distribution: norm mean:
    127.751
    std: 0.003
std:
distribution: norm mean:
    1.734
std: 0.189
B:
avg:
distribution: norm mean:
    127.867
```

```

    std: 0.03
    std:
distribution: norm mean:
    1.534
std: 0.157
L:
avg:
distribution: norm mean:
    241.621
    std: 0.538
    std:
distribution: norm mean:
    13.346
    std: 0.177
color_space: LAB
methods: Reinhard
n_each_class: 0
random: true

```

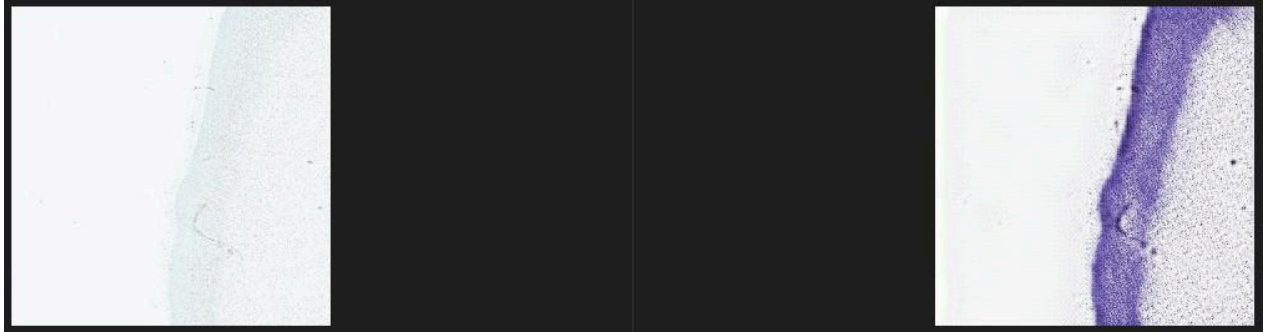
The methods parameter refers to the normalization technique to be used for the whole process.

These configuration files are then used to perform the aug+norm operations on the dataset. This results in a dataset with visibly changed images which I then used to train the stainGAN with self attention as in section 8 above. The intermediate training results are below :



11. Vanilla StainGAN and aug+norm

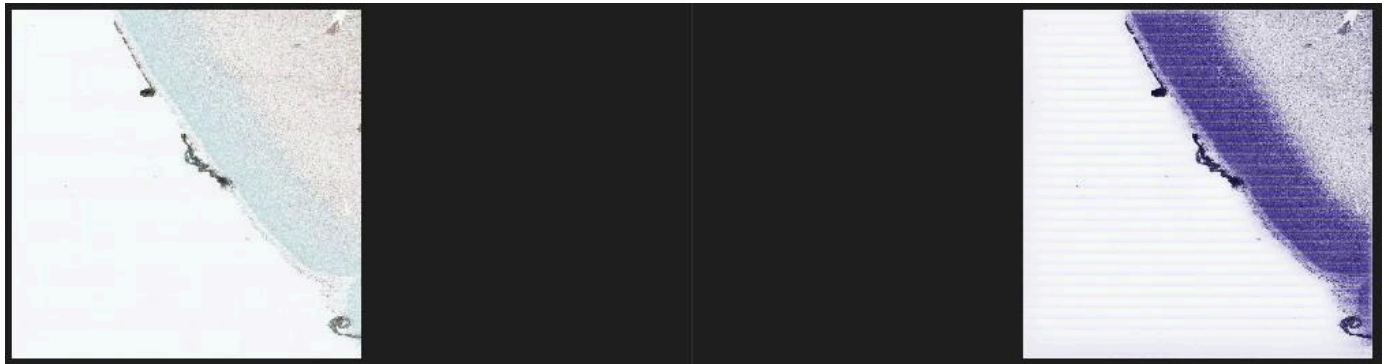
We decided to try training the vanilla stainGAN (ResNet Generators, PatchGAN discriminators, neither with self attention) on the augmented and normalized dataset, and the results weren't too bad :



The image on the left is blurry because of the aug+norm. I didn't do inference until later because the normalization done during preprocessing for training and the normalization done during inference should ideally be the same, and that took me a while to figure out.

12. stainGAN with UNet Generators and PixelGAN discriminators

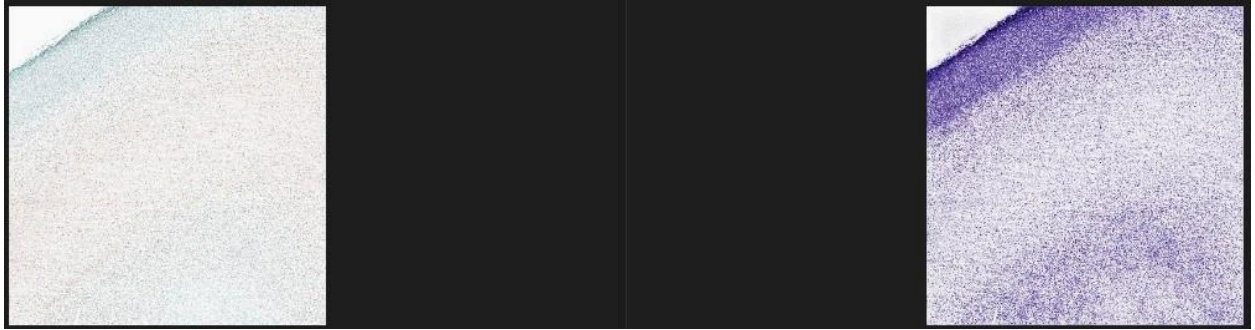
I tried to change the architecture to a UNet just for experimentation, and I used the PixelGAN discriminator with self attention and instance normalization. A PixelGAN is just like a PatchGAN but instead classifies pixels and not a region of an image. I tried using a PixelGAN with self attention but ran into memory problems again.



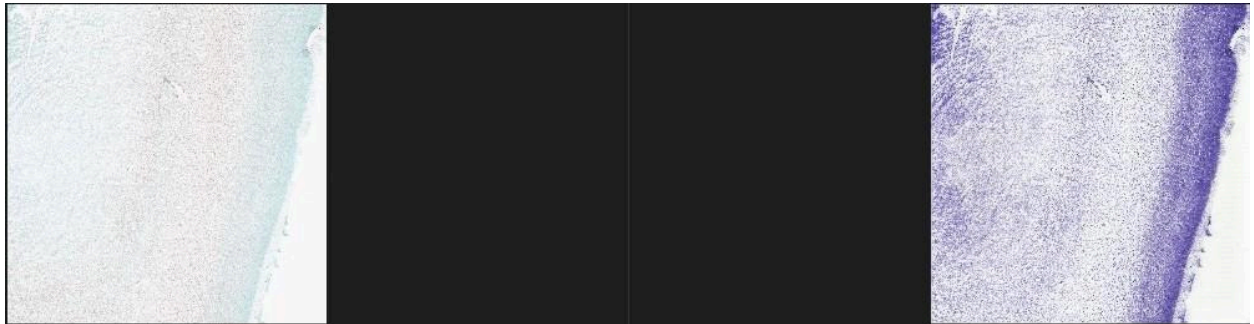
13. stainGAN with UNet Generators and PatchGAN discriminators with self

attention I did two experiments, one with aug+norm and the other without aug+norm.

Without aug+norm :

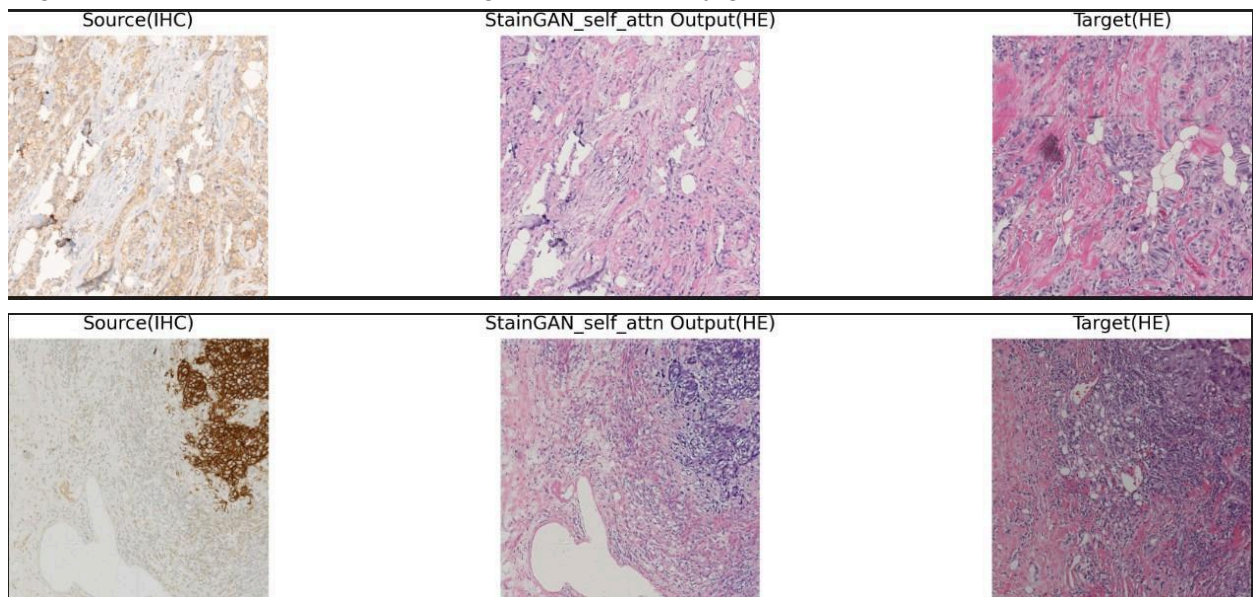


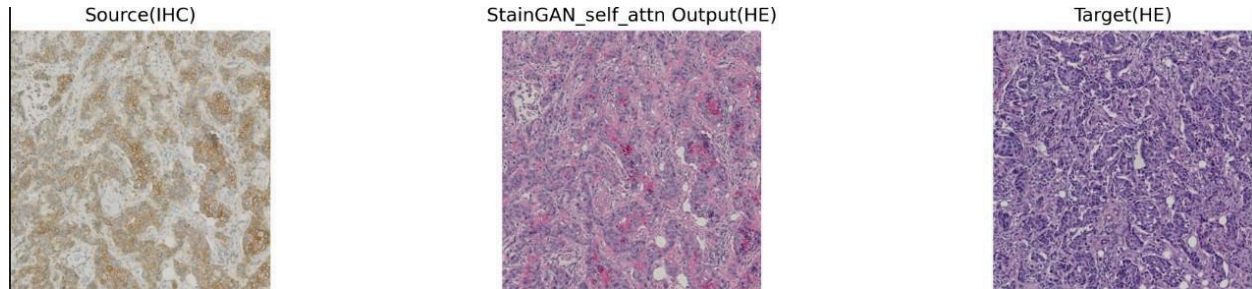
With aug+norm:



14. stainGAN with UNet generators and PatchGAN discriminators with self attention on type_3 BCI patches

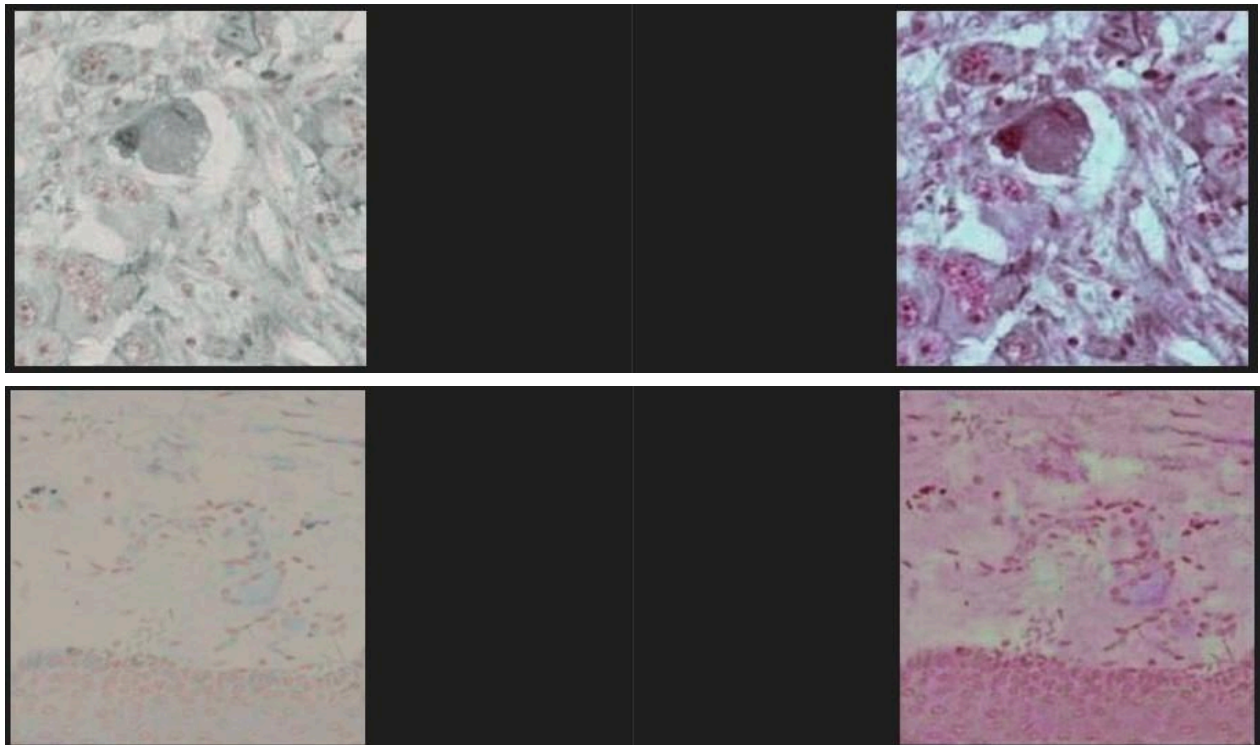
Before combining the full BCI dataset we decided to train the model on type_3 images without aug+norm. These are the results (targets aren't exactly ground truth) :





15. stainGAN with UNet generators and PatchGAN discriminators with self attention on all BCI patches

We trained the model on the whole dataset without realising how overrepresented every non 0 type was :

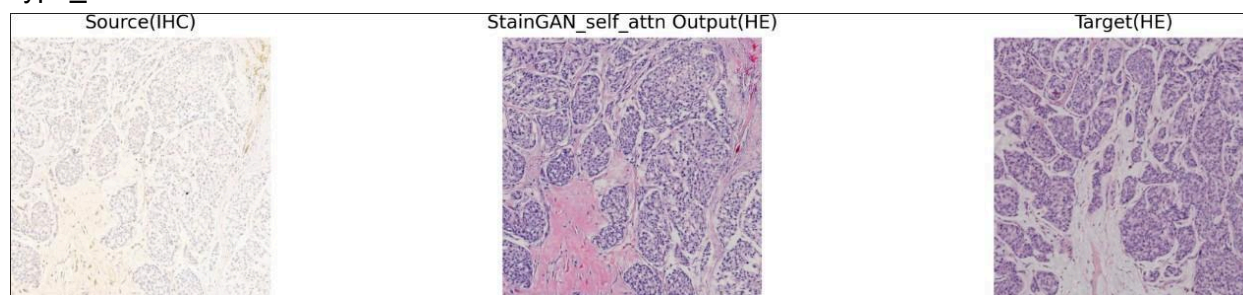


So I had to painstakingly make the dataset more representative. And we trained the model again.

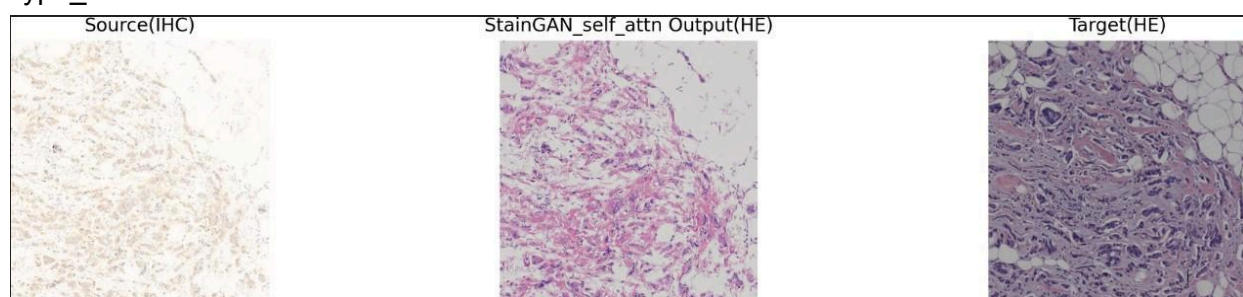
16. stainGAN with UNet generators and PatchGAN discriminators with self attention on all BCI patches(representative)

For some reason I couldn't find the image comparisons for the model trained with augmentation and normalization. But here are the comparisons for the model trained without aug+norm:

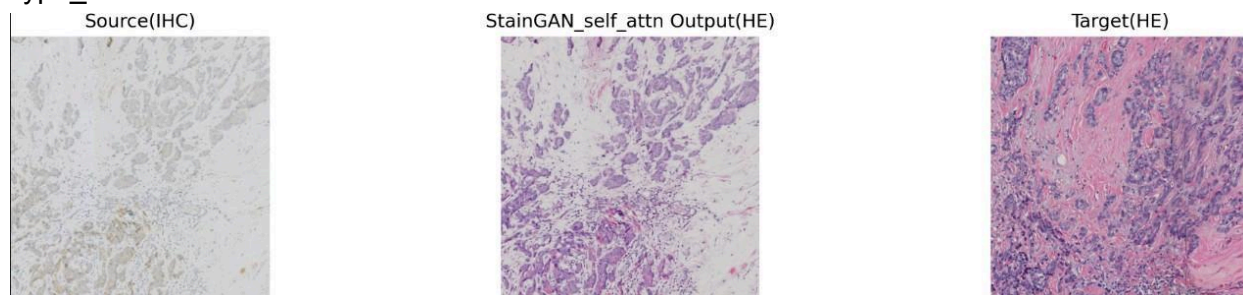
Type_0 :



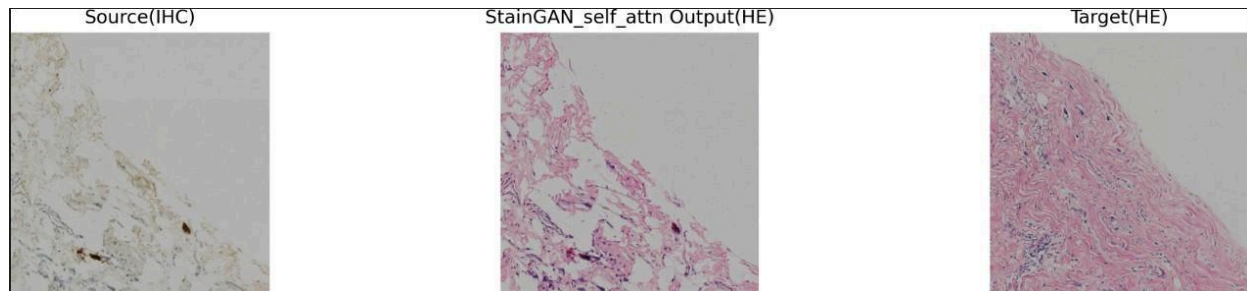
Type_1 :



Type_2:



Type_3 :



17. Using the previous model on the ALLEN dataset

We decided to test out the model from the previous section that was trained on the representative dataset with augmentation and normalization.

Here's the code that I used to normalize the images during inference, I had to use the same normalization techniques that was used before training and during preprocessing..

```
from aug_norm import randstainna
def norm_randstainna(source : np.ndarray,target : np.ndarray,yaml_path):
    randstainna_ = randstainna.RandStainNA(
yaml_file = yaml_path, std_hyper =
        0.0, distribution =
        'normal', probability =
        1.0, is_train = False
    )
    source_avg,source_std=randstainna_._getavgstd(source)#returns a tuple of
np.arrays containing avg and std of the image
target_avg,target_std=randstainna_._getavgstd(target)

    return
    randstainna_._normalize(source,source_avg,source_std,target_avg,target_std)

def un_norm_randstainna(source : np.ndarray,target :
np.ndarray,yaml_file_path):
    """
    Here source should be the target image passed to norm_randstainna and the
target should be the input/source image passed to norm_randstainna
    """
    randstainna_ = randstainna.RandStainNA(
        yaml_file = yaml_file_path,
        std_hyper = 0.0,
distribution = 'normal',
```

```

probability = 1.0, is_train =
    False
    )
    source_avg,source_std=randstainna._getavgstd(source)#returns a tuple of
    np.arrays containing avg and std of the image
    target_avg,target_std=randstainna._getavgstd(target)

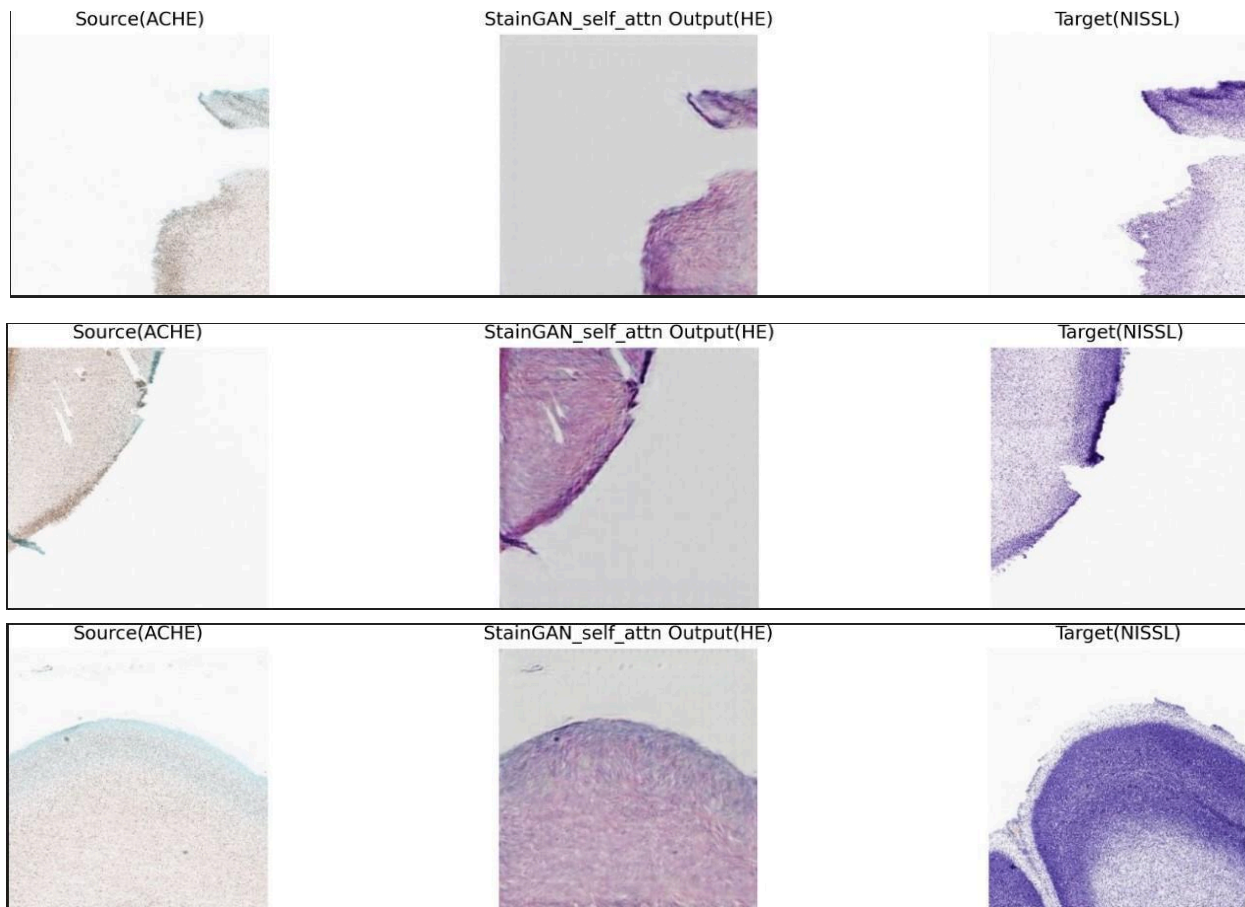
    return
    randstainna._normalize(source,source_avg,source_std,target_avg,target_std)

def stainGANEval(model,source):
    model.eval()
    with torch.no_grad():
        source_norm=norm(source)
        img_gan=model(source_norm.cuda())
        img_gan=un_norm(img_gan)
# plt.imshow(img_gan)
        # plt.title('StainGAN generated output(ground truth for stainNet)')
    return img_gan
def stainGAN_eval_aug_norm(model,source:np.ndarray,target :
    np.ndarray,yaml_file_path : str):
    """
        The yaml_file_path doesn't matter, during normalization, only the
        color_space is taken from the yaml_file, which in the case of IHC and HE images
        are the same : LAB
    """
    model.eval()
    with torch.no_grad():
        source_norm=norm_randstainna(source,target,yaml_file_path)
        img_gan=model(torch.from_numpy(source_norm).cuda())
        img_gan=un_norm_randstainna(target,source,yaml_file_path)

    return img_gan

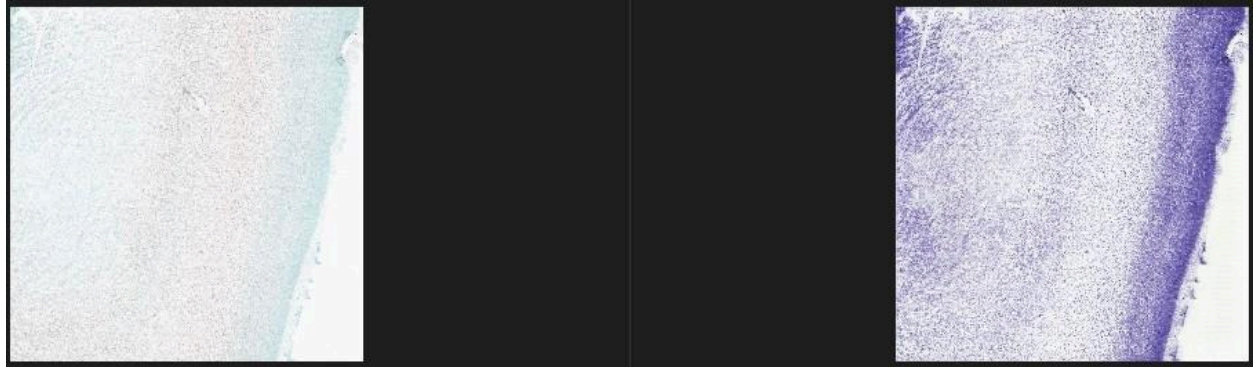
```

Since the model trained on BCI dataset can only convert an IHC slice to an HE slice and vice versa, we expected it to give a bad output for the allen dataset, because it has just Ache and NISSL stains. But it turned out to work pretty well. There are still improvements that can be made though.



18. stainGAN with UNet generator and PatchGAN(with self attention) discriminator trained on allen dataset

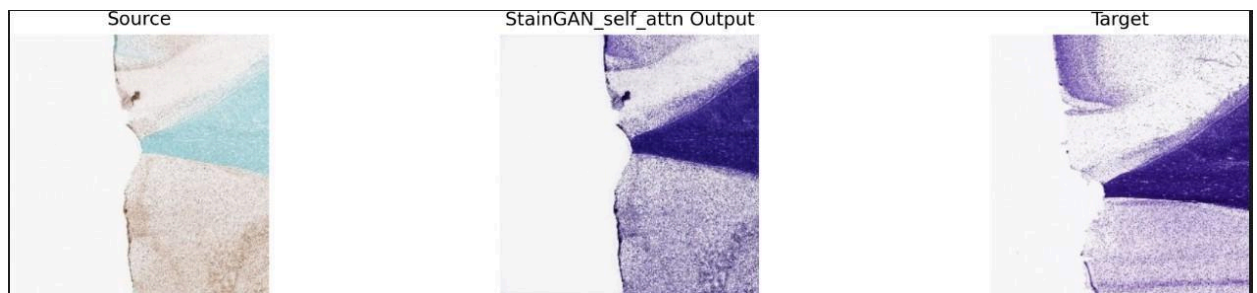
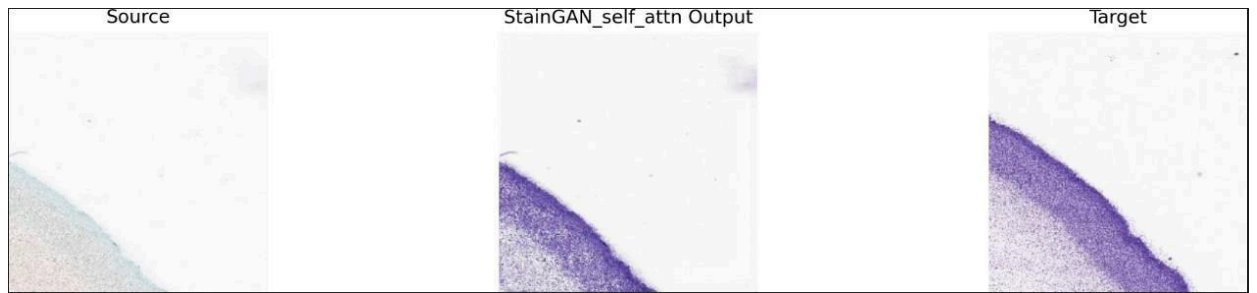
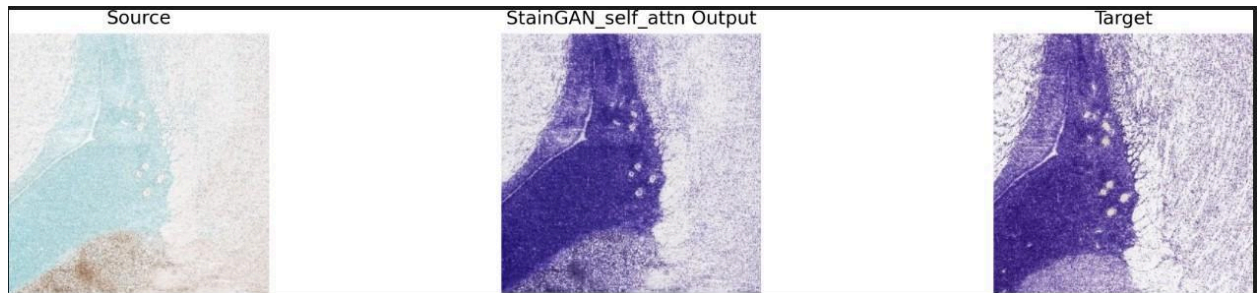
This is the final model we trained, and by far has shown the best results, unfortunately there seems to be a problem with the inference I tried to do on the model that was trained on the dataset with augmentation and normalization. Probably something to do with the last code snippet above wherein I tried to isolate the functionality of normalization from the combination of augmentation and normalization. But the intermediate training images from the last epoch seem to have a very good output:

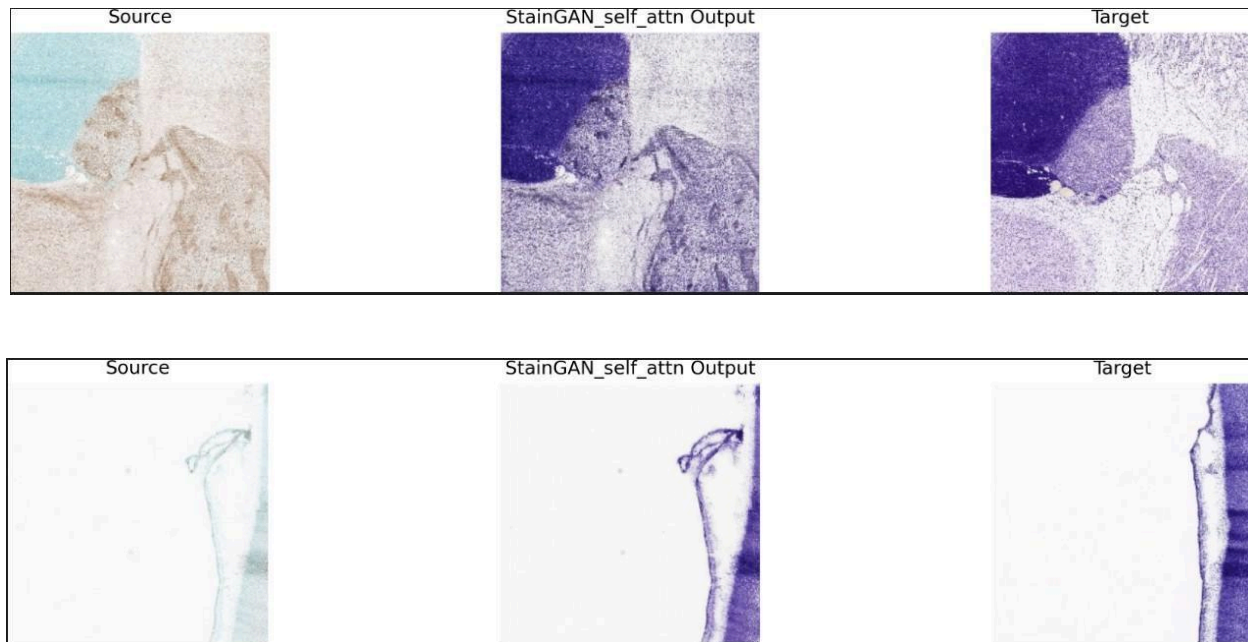


Augmented Ache

Model output

The results came through very well too for the model trained on the normal dataset **without** augmentation and normalization :





FUTURE IMPROVEMENTS

Here are some possible improvements and avenues to explore to better the model

1. Different discriminators

I was unfortunately pressed for time and couldn't fully explore the papers that seemed to present good results for tasks related to image to image translation, like the possibility of using discriminators with different architectures.

2. Different attention mechanisms

I have only experimented with the scaled dot product self attention layers in this project so far. I would love to explore other promising architectures like [CBAM](#) and even using a [vision transformer in the architectures](#).

3. Different Loss Functions

Some papers we stumbled upon during and after making out latest successful model are : [Adaptive Supervised NCE Loss](#), [Pathological Loss](#)

4. Exploring the [stanford paper](#) more

I would like to explore the methods elucidated in the paper more, and tinker with the architectures and spectral normalization.

5. Synthesize images for the BCI dataset

This is what we planned to do after the latest model, this [paper](#) tries to synthesize new images for the BCI dataset using a Pix2Pix model, which was the precursor to the CycleGAN model (which our models are based on). Pix2Pix uses supervised learning to do the same task CycleGAN does but with paired image data. So it is fed images of both domains(stains) together to be able to learn a mapping to transform an input image to the target domain.

FINAL WORDS

I am strongly of the opinion that this project holds immense significance for SGBC and CCB, a successful and robust model that may even be online(continuous training even when deployed and in use) could be a game changer, reducing costs and saving a lot of time for the researchers involved in using stained brain slices.